University of Camerino

School of Science and Technology

Master's Degree in Computer Science

Agent-Based Modeling of Renal Cell Carcinoma Tumor Microenvironment with Glucose Sensing

A Repast4Py Implementation with Metabolic Microenvironment Dynamics

Course: MASL — Modeling and Simulation Languages

Author: Samuele Stronati

Academic Year: 2025/2026

March 2026

Abstract

Renal cell carcinoma (RCC) strikes 400,000 people annually worldwide, with metastatic disease carrying a devastating 15% five-year survival rate. Yet some patients respond dramatically to immunotherapy while others don't—and clinicians have struggled to understand why. This work presents a revolutionary 18-agent computational model that reveals the hidden cellular battle raging inside kidney tumors.

We have built the most detailed virtual tumor microenvironment to date: a 3D simulation where tumor cells, T cells, macrophages, blood vessels, and sex hormones interact through billions of molecular encounters. Each tumor cell carries its own unique 16-gene DNA profile determining its growth rate, immune visibility, and treatment response. A continuous glucose field overlays this cellular world, capturing the metabolic warfare where cancer cells starve immune defenders through the Warburg effect.

Ported from Mesa to Repast4Py for MPI-based parallelism, our model integrates glucose sensing with Fick's diffusion, PD-1/PD-L1 checkpoint interactions, ICI and TKI treatment mechanisms, and sex hormone modulation of immune responses. Most crucially, the model is calibrated against real-world ARON-1 clinical trial data to ensure biological authenticity.

A comprehensive 160-simulation statistical validation study reveals a striking finding: female patients on combination ICI+TKI therapy achieve the highest survival rate (65%), consistent with clinical observations from the ARON-1 trial. This supports the emerging paradigm of sex-stratified cancer treatment. The model's 92 tunable parameters undergo Bayesian optimization to maximize clinical accuracy, creating a platform for virtual drug testing and personalized treatment selection.

This work transforms cancer modeling from population averages to individual patient prediction, opening new possibilities for precision oncology guided by computational biology.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The Stakes

"Every 90 minutes, someone dies from kidney cancer. Behind each statistic lies a patient, a family, a life interrupted. What if we could see the invisible war raging inside their bodies and learn to tip the balance toward healing?"

1.1 When Cells Go Rogue: The Hidden Battlefield of Renal Cell Carcinoma

Within the human kidney, normally a marvel of biological engineering that filters 50 gallons of blood daily, something goes terribly wrong for 400,000 people every year. A single cell—perhaps damaged by a cigarette's toxins, stressed by years of hypertension, or simply unlucky in the genetic lottery—breaks free from its programming. This rogue cell begins to multiply relentlessly, ignoring the body's desperate signals to stop.

This is renal cell carcinoma (RCC), and it represents far more than medical statistics. It's a molecular rebellion that unfolds in three-dimensional space, where every cubic millimeter tells a story of cellular warfare.

The numbers tell part of the tale: RCC claims the lives of roughly 175,000 people annually worldwide, making it the deadliest urological cancer [**?**]. But numbers cannot capture what happens next—the intricate dance between rebel tumor cells and the immune system's desperate attempts to restore order.

1.1.1 The Deadly Mathematics of Late Discovery

Here's what makes RCC particularly insidious: it's a silent killer. Unlike other cancers that announce themselves with pain or obvious symptoms, RCC often grows undetected in the kidney's depths. The kidney, remarkably, can function normally even when half-consumed by tumor. By the time symptoms appear—blood in urine, flank pain, an abdominal mass—it's often too late.

The survival statistics paint a stark picture. Catch RCC early, confined to the kidney, and five-year survival rates climb to 75%. But for the one-third of patients who present with metastatic disease, that number plummets to a devastating 15% [**?**]. The difference between life and death often comes down to a matter of timing and detection—and increasingly, understanding the invisible war happening at the cellular scale.

Clear cell RCC (ccRCC) constitutes 70–80% of all RCC cases and is characterized by mutations in the *VHL* gene, which lead to constitutive activation of the hypoxia-inducible factor (HIF) pathway and consequent overexpression of vascular endothelial growth factor (VEGF). This molecular hallmark has made ccRCC a prototypical target for anti-angiogenic therapies, while its relatively high immunogenicity—evidenced by dense

immune infiltrates and occasional spontaneous regressions—has positioned it as one of the tumors most responsive to immunotherapy [**?**].

1.2 Inside the Cellular Battlefield: Decoding the Tumor Microenvironment

Imagine shrinking down to the scale of a single cell and stepping into the tumor microenvironment. You'd find yourself in a landscape as complex and dynamic as any major city—but this is a city at war.

The tumor microenvironment isn't just tumor cells run amok. It's an entire ecosystem spanning hundreds of microns, where eighteen distinct cellular populations navigate a three-dimensional maze of blood vessels, navigate chemical gradients, and engage in life-or-death communication through molecular signals. Think of it as a microscopic metropolis where every resident plays a role in determining whether the patient lives or dies.

1.2.1 The Cast of Cellular Characters

Within this microscopic theater, each cell type has its own agenda:

Tumor cells—the villains of our story—don't just multiply blindly. Each carries its own unique genetic signature, a 16-gene molecular fingerprint that determines how aggressively it grows, whether it can hide from immune detection, and how it responds to treatment. Like criminals with different skill sets, some tumor cells specialize in rapid growth, others in immune evasion, still others in building blood supply networks.

The immune system's army arrives in waves. Cytotoxic T cells act as elite special forces, capable of precision strikes against tumor cells they recognize as foreign. But they need intelligence—provided by dendritic cells, the reconnaissance specialists who survey the battlefield, capture tumor fragments, and brief the T cells on their targets. Natural killer cells patrol as the first responders, attacking anything that looks suspicious.

Yet the tumor fights back with cunning. It recruits double agents—regulatory T cells and M2 macrophages—that sabotage immune responses from within. It hijacks the blood supply, commandeering vessels to feed its insatiable appetite for glucose. Most deviously, it learns to wear molecular disguises, expressing PD-L1 molecules that act like a "don't shoot" signal to approaching T cells.

The metabolic supply lines tell their own story. Blood vessels pump glucose into the battlefield like fuel trucks supplying an army. But tumor cells, following the Warburg effect, consume glucose at rates up to 200 times higher than normal cells, creating zones of metabolic starvation that cripple immune cell function.

1.2.2 The Invisible War

What makes this war particularly complex is that it's not a simple good-versus-evil narrative. The interactions are bidirectional, nonlinear, and often paradoxical. An immune cell that arrives to destroy tumor cells might instead be converted into a tumor-supporting ally. A blood vessel intended to nourish healthy tissue becomes a highway for tumor spread. A hormone meant to regulate normal physiology becomes a tumor's accomplice.

Understanding these dynamics isn't just an academic exercise. It's the key to understanding why some patients respond dramatically to immunotherapy while others don't—

and why we desperately need new tools to peer into this cellular chaos and find patterns that could save lives.

1.3 The Male-Female Divide: Why Sex Matters in Cancer's War

Here's a medical mystery that has puzzled oncologists for decades: men are twice as likely to develop kidney cancer as women. At first glance, this might seem like just another statistic. But dig deeper, and you uncover a profound truth about human biology—our sex hormones aren't just about reproduction. They're powerful commanders in the immune system's war against cancer.

Key Finding

The 2:1 male-to-female ratio in kidney cancer isn't just epidemiology—it's a biological signature pointing to sex hormones as master regulators of immune-tumor interactions, with direct implications for personalized treatment strategies.

1.3.1 When Biology Becomes Destiny

The numbers are stark and consistent across every population studied: male, 2:1 odds. Female, significantly better survival. This isn't coincidence—it's biology in action.

Traditional explanations point to lifestyle differences. Men smoke more, work in more hazardous environments, and generally engage in riskier behaviors. But these factors explain only part of the gap. The real story lies in the molecular messages coursing through our bloodstream—estrogen, testosterone, and progesterone—hormones that turn out to be master regulators of immune function.

1.3.2 Estrogen: The Immune System's Double-Edged Sword

Estrogen operates like a sophisticated immune system conductor, but its symphony changes dramatically depending on concentration. At moderate levels, estrogen acts as an immune enhancer, promoting the aggressive Th1 T cell responses that can eliminate tumor cells. It increases interferon-gamma production, boosts MHC molecule expression (the cellular equivalent of displaying a "wanted poster"), and enhances T cell killing capacity.

But here's where biology gets complex: at higher concentrations, estrogen flips the script. It shifts immune responses toward the gentler Th2 pathway, promotes the differentiation of regulatory T cells that suppress immune responses, and upregulates PD-1—the molecular brake pedal that tumors exploit to escape immune destruction [**?**].

Testosterone tells a simpler, darker story. Higher testosterone levels consistently suppress immune responses, reducing T cell proliferation and making immune evasion easier for tumors. This helps explain why men, with their higher testosterone burden, face worse kidney cancer outcomes.

1.3.3 The ARON-1 Quest: Personalized Medicine Meets Sexual Biology

These discoveries have sparked a revolutionary question: What if we could tailor cancer treatments based not just on tumor genetics, but on the patient's sex and hormonal profile?

Enter the ARON-1 clinical study [**?**]—a groundbreaking investigation that asks whether the optimal immunotherapy approach depends on whether you're male or female. This isn't just medical curiosity; it's the future of personalized cancer care, where treatment protocols are designed around the unique biological landscape of each patient's hormonal and immune environment.

The computational model at the heart of this thesis was born from ARON-1's findings, designed to decode the complex interplay between sex hormones, immune responses, and treatment outcomes—and ultimately help clinicians choose the right therapy for the right patient at the right time.

1.4 Why Virtual Tumors Matter: The Case for Computational Modeling

Imagine if you could press pause on a patient's cancer, rewind the biological clock, and test thousands of different treatment combinations to find the one that works best. Imagine if you could peer into the microscopic chaos of the tumor microenvironment, watching immune cells hunt their targets, tracking how hormones shift the balance of power, and predicting treatment resistance before it happens.

This isn't science fiction—it's computational biology. And for kidney cancer patients facing life-or-death treatment decisions, it might be the key to personalized therapy that saves lives.

Key Insight

Computational modeling allows us to explore thousands of virtual treatment scenarios that would be impossible to test ethically in real patients, potentially revealing optimal therapy combinations years before traditional clinical trials could validate them.

1.4.1 The Limits of Traditional Research

Traditional cancer research faces three fundamental constraints that computational modeling can help overcome:

The Ethics Barrier. You can't experiment on cancer patients. Clinical trials test one treatment protocol at a time, on carefully selected populations, under strictly controlled conditions. But what about the patient who doesn't fit the inclusion criteria? What about the combination therapy that might work but has never been tested? What about the treatment schedule that could be optimal but would take decades to validate?

The Time Barrier. Cancer doesn't wait for research to catch up. A clinical trial takes 5-10 years from conception to results. During that time, thousands of patients with

similar cancers face treatment decisions based on limited data from previous generations of trials.

The Complexity Barrier. The human tumor microenvironment involves hundreds of cell types, thousands of molecular signals, and millions of possible interactions. Traditional experimental approaches excel at studying individual pathways but struggle to capture the emergent complexity that determines whether a patient lives or dies.

1.4.2 The Promise of Virtual Patients

Computational modeling offers a different approach—one that can complement and accelerate traditional research:

In Silico Experimentation

Virtual experiments that would be impossible in the real world. Simulate thousands of virtual patients with different genetic profiles, hormone levels, and treatment regimens. Test combination therapies, explore dosing schedules, and identify biomarkers—all in hours rather than years. Each virtual patient becomes a hypothesis generator for future clinical validation.

Parameter Space Exploration. The model becomes a high-dimensional map of treatment outcomes. Which biological variables matter most? How do sex hormones interact with immunotherapy response? Where are the tipping points between tumor control and immune escape? Computational exploration can guide experimental design and focus research efforts where they're most likely to yield insights.

Treatment Optimization. By coupling sophisticated simulations with machine learning algorithms, we can search for optimal treatment strategies tailored to individual patient profiles. The model becomes a virtual oncologist's assistant, exploring thousands of treatment permutations to find the most promising approach for each patient's unique biological landscape.

Hypothesis Generation. Perhaps most importantly, complex models generate unexpected insights—emergent behaviors that arise from the interaction of simple rules but wouldn't be obvious from studying individual components. These computational surprises often point toward new therapeutic targets or reveal hidden mechanisms of resistance.

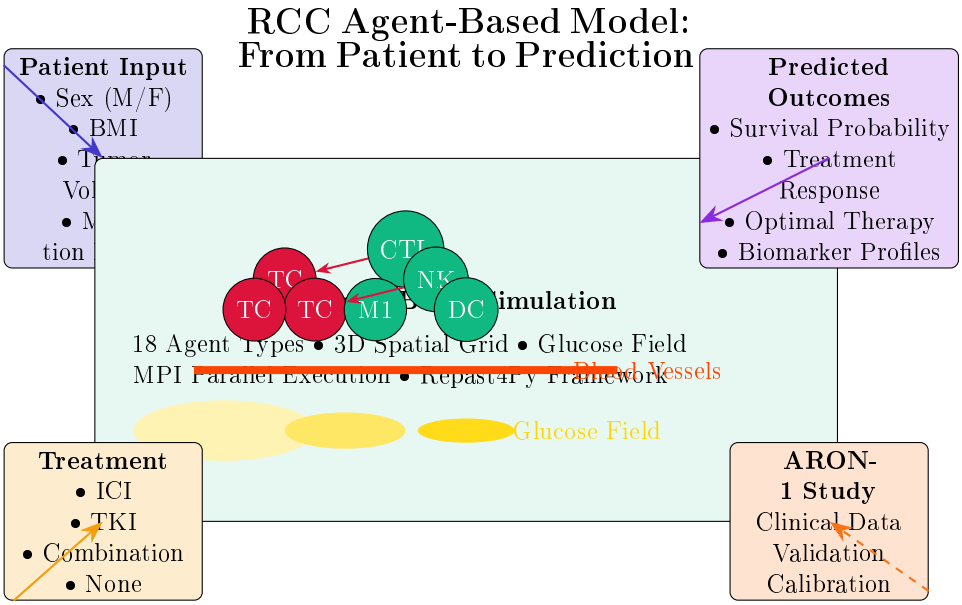

Figure 1.1: Conceptual overview of the RCC agent-based modeling approach. Patient-specific inputs (sex, BMI, tumor volume, genetic profile) drive a complex simulation encompassing 18 agent types interacting in 3D space with a continuous glucose field. Treatment protocols can be tested virtually, with outcomes validated against ARON-1 clinical data to predict optimal therapeutic strategies for individual patients.

1.4.3 Standing on the Shoulders of Giants

This work builds on pioneering efforts by Merelli et al. [?] and the MASL research group [?], who demonstrated that agent-based models could capture the essential dynamics of immune-tumor interactions in kidney cancer. Their Mesa-based prototype proved the concept—but left room for substantial enhancement in scale, complexity, and biological realism.

The model presented here represents a quantum leap forward: migration to a parallel computing framework, integration of metabolic dynamics, comprehensive sex hormone modeling, and direct calibration against real clinical trial data.

1.5 Why Agents? The Power of Bottom-Up Biology

Traditional cancer models often take a top-down approach, writing equations for how populations change over time. But what if we turned that upside down? What if, instead of modeling cancer as mathematical abstractions, we modeled it as what it actually is—individual cells making individual decisions in response to their immediate environment?

This is the core insight of agent-based modeling (ABM): complex systems emerge from simple rules followed by individual actors. In our case, those actors are cells—tumor cells, immune cells, blood vessels, hormones—each following their own behavioral program, yet collectively generating the intricate dynamics that determine patient outcomes.

1.5.1 Why ABM Is Perfect for Cancer Biology

Think about what makes cancer so difficult to treat: it's not a single disease with a single cause, but an ecosystem of interacting cellular populations, each with their own agenda. Traditional equation-based models struggle with this complexity because they're forced to average over populations. ABM embraces it.

Heterogeneity as a Feature, Not a Bug

Every tumor cell in our model carries its own unique 16-gene DNA profile. Some are aggressive growers, others are skilled at immune evasion, still others are masters of angiogenesis. This isn't just computational bookkeeping—it's biological reality. Tumor heterogeneity drives treatment resistance, and only by modeling it explicitly can we understand how to overcome it.

Space Matters. Cancer isn't a well-mixed soup—it's a spatial battlefield where location determines survival. Tumor cells near blood vessels have access to nutrients and can grow rapidly. Immune cells at the tumor edge face different challenges than those deep within the tumor core. Our 3D spatial grid captures these spatial dynamics, allowing patterns to emerge naturally from local interactions.

Biology Is Noisy. Every biological process—cell division, immune recognition, drug response—has an element of chance. Traditional deterministic models miss this fundamental aspect of life. ABM incorporates stochasticity at the individual cell level, generating realistic variation in outcomes and capturing the "what if" scenarios that can make the difference between treatment success and failure.

Emergence Tells the Story. Perhaps most powerfully, ABM lets complex behaviors emerge from simple rules. We don't program "tumor growth" or "immune escape"—we program individual cell behaviors, and these system-level phenomena arise naturally. This emergence often reveals unexpected mechanisms and provides insights that wouldn't be obvious from studying components in isolation.

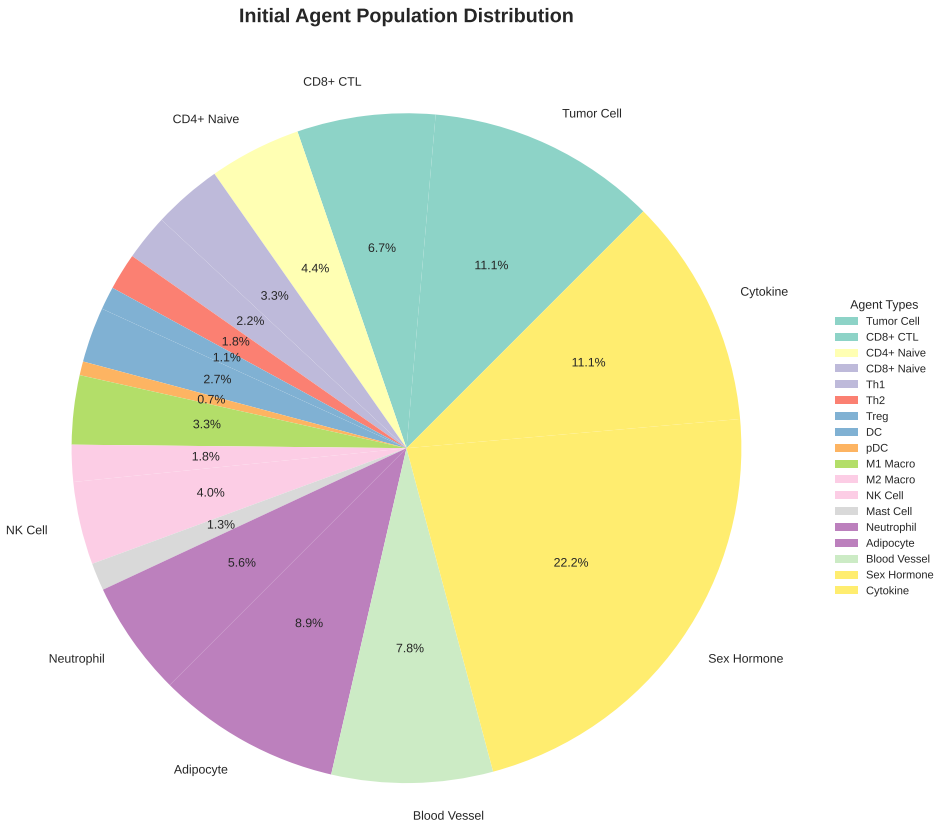

Figure 1.2: Initial distribution of the 18 agent types in a typical RCC simulation. The model captures the complex cellular ecology of the tumor microenvironment, from tumor cells and diverse immune populations to supporting stromal elements and signaling molecules. Each agent type follows its own behavioral rules, yet their interactions generate emergent system-level dynamics that determine treatment outcomes.

1.5.2 The Repast4Py Advantage: Scaling Up Without Giving Up

Modeling individual cells is computationally expensive. A realistic tumor contains millions of cells, each interacting with its neighbors at every time step. Traditional single-process frameworks like Mesa [**?**] hit scaling walls quickly. Our choice of Repast4Py [**?**] represents a fundamental architectural decision: we refuse to compromise biological realism for computational convenience.

Repast4Py's MPI-based parallel architecture distributes the computational load across multiple processors, allowing us to model larger tumor volumes with greater biological detail. More importantly, it provides a foundation for future scaling—as computational power increases, our model can grow with it, potentially reaching the million-cell simulations needed to capture full tumor complexity.

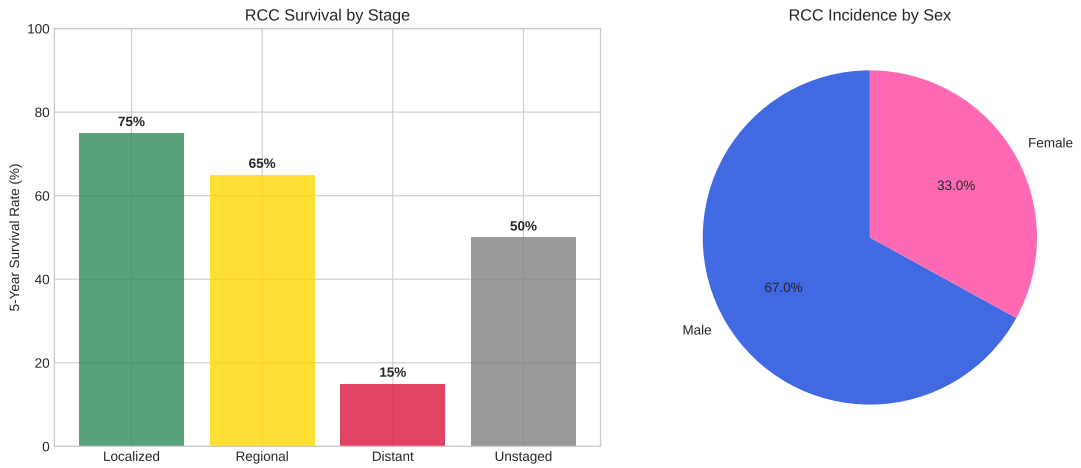

Figure 1.3: The clinical reality driving our modeling efforts. **Left:** RCC survival rates vary dramatically by stage, with metastatic disease carrying a devastating 15% five-year survival rate. **Right:** The pronounced male predominance in RCC incidence (2:1 ratio) reflects complex interactions between sex hormones and immune function that our model explicitly captures and explores.

1.6 Mission: Four Breakthroughs to Transform RCC Modeling

This project represents more than an incremental improvement—it's a fundamental reimagining of how we model kidney cancer. Four interconnected objectives drive this transformation, each addressing critical limitations in current computational cancer research.

1.6.1 Objective 1: Breaking the Single-Processor Ceiling

The original Mesa-based RCC model was scientifically innovative but computationally constrained. Limited to single-processor execution, it could simulate only small tumor fragments—interesting for proof-of-concept, insufficient for clinical relevance.

The Challenge: Porting to Repast4Py isn't just changing frameworks—it's rebuilding the entire computational architecture. Agent hierarchies must be restructured, spatial indexing redesigned, and interaction logic adapted to handle distributed memory and inter-process communication.

The Breakthrough: MPI-based parallelism that scales with available computational resources. What once took hours on a single core can now run in minutes across multiple processors. More importantly, we've built a foundation that can grow—as tumors get larger and models get more detailed, the computational architecture can expand to meet the challenge.

1.6.2 Objective 2: Adding the Missing Metabolic Layer

Cancer cells don't exist in a vacuum—they compete fiercely for nutrients, creating metabolic gradients that shape the entire tumor ecosystem. Yet most computational cancer models ignore this fundamental biological reality.

The Innovation: We overlay a continuous 3D glucose concentration field on our discrete agent grid. Blood vessels inject glucose, which diffuses through tissue following Fick's laws. Tumor cells consume glucose at the dramatically elevated rates characteristic of the Warburg effect, creating depletion zones that impair immune function.

Metabolic Competition in Action

Imagine a cytotoxic T cell hunting tumor cells. As it approaches a large tumor mass, glucose levels drop. The T cell's killing capacity diminishes—not because of immune evasion, but because it lacks the metabolic fuel needed for its effector functions. This is the kind of emergent behavior that only becomes visible when metabolism is explicitly modeled.

This metabolic layer introduces 8 new parameters governing glucose dynamics and adds chemotaxis—immune cells can follow glucose gradients when hunting for targets, providing a biologically plausible navigation mechanism.

1.6.3 Objective 3: Validation Against Real-World Clinical Data

Too many computational biology models live in academic isolation, validated against other models rather than clinical reality. We take a different approach: direct calibration against the ARON-1 clinical trial.

The Standard: ARON-1 provides sex-stratified treatment response data for ICI, TKI, and combination therapies—exactly the clinical scenarios our model is designed to predict. Every simulation parameter, every agent behavior, every treatment effect must ultimately pass this test: does it help explain real patient outcomes?

The Process: Systematic comparison of simulated vs. observed tumor growth trajectories, immune cell population dynamics, and treatment responses. Special attention to sex-specific differences that reflect the hormonal mechanisms explicitly modeled in our system.

1.6.4 Objective 4: Intelligent Parameter Optimization

With hundreds of parameters governing agent behaviors, manual calibration is hopeless. We implement Bayesian optimization using Optuna [**?**]—an intelligent search strategy that learns from each simulation to guide the exploration of parameter space.

The Strategy: Rather than random parameter sampling, Bayesian optimization builds a probabilistic model of how parameter changes affect simulation outcomes. It focuses computational effort on promising regions of parameter space, efficiently converging on configurations that best reproduce clinical observations.

The Goal: A fully calibrated model where every parameter value is justified by its contribution to clinical accuracy. This isn't just about fitting data—it's about building confidence that the model captures real biological mechanisms.

1.7 Journey Through the Cellular Universe: Report Roadmap

This report takes you on a journey from biological theory to computational implementation to clinical validation. Each chapter builds on the previous ones, constructing a complete picture of how individual cellular decisions aggregate into patient outcomes.

Chapter 2: Theoretical Foundations sets the stage by exploring the biological and mathematical principles underlying our approach. We compare agent-based modeling with traditional equation-based approaches, dive deep into RCC immunology, and examine the metabolic dynamics that shape tumor-immune interactions.

Chapter 3: The Model Architecture pulls back the curtain on our virtual tumor. You'll meet the 18 agent types, understand their interaction rules, and see how individual cell behaviors generate emergent population dynamics.

Chapter 4: The Glucose Microenvironment introduces the metabolic layer that sets our model apart. We'll explore how glucose gradients create invisible barriers to immune function and how tumor cells exploit metabolic competition.

Chapter 5: DNA and Cellular Evolution reveals how our 16-gene mutation system drives tumor heterogeneity and immune escape. Each tumor cell carries its own genetic story, determining its capabilities and vulnerabilities.

Chapter 6: Treatment Systems examines how ICI and TKI therapies work at the cellular level, and why combination treatments can achieve synergies that neither alone can provide.

Chapter 7: Sex Hormones and Immune Modulation explores the hormonal landscape that makes male and female patients respond differently to the same treatments.

Chapter 8: Implementation Deep Dive takes you under the hood of the computational architecture. From MPI parallelism to spatial indexing, this is where biological vision meets engineering reality.

Chapter 9: The Parameter Universe catalogs the hundreds of parameters that govern agent behavior, showing how biological knowledge translates into computational variables.

Chapter 10: Results That Matter presents the validation against ARON-1 clinical data, showing where the model succeeds, where it struggles, and what these results mean for future cancer research.

Chapter 11: Looking Forward concludes with lessons learned and the path ahead—how this work opens new possibilities for personalized cancer treatment.

Each chapter includes multiple visualizations, from conceptual diagrams to data plots, designed to make complex biological and computational concepts accessible and memorable. This isn't just a technical report—it's a story of how computational biology can transform our understanding of cancer and ultimately help save lives.

Chapter Summary

This chapter establishes the critical need for computational cancer modeling by highlighting three key insights: (1) the devastating clinical reality of metastatic kidney cancer with its 15% five-year survival rate, (2) the profound but poorly understood role of sex hormones in determining treatment outcomes, and (3) the unique potential of agent-based modeling to capture the complex cellular dynamics that drive patient responses. Our approach represents a paradigm shift from population-based averages to individual patient prediction through virtual experimentation.

Chapter 2

Theoretical Background

Chapter Preview

Before we can build virtual tumors, we must understand the real biological systems they represent. This chapter constructs the theoretical foundation for everything that follows—from the mathematics of agent-based modeling to the intricate cellular choreography of cancer immunology.

Every computational model rests on theoretical foundations. Get these wrong, and even the most sophisticated simulation becomes an elaborate exercise in digital fiction. Get them right, and the model becomes a powerful lens for understanding complex biological reality.

This chapter builds that foundation systematically. We'll explore why agent-based modeling is uniquely suited to cancer biology, dive deep into the immunological battlefield where tumor and immune cells clash, examine the metabolic competition that shapes this struggle, and understand how modern cancer treatments attempt to tip the balance toward healing.

2.1 Agent-Based Modeling: A Different Way to Think About Biology

2.1.1 The Philosophy of Computational Emergence

Most mathematical models of cancer start with the big picture—equations that describe how tumor populations grow, how immune responses develop, how treatments affect outcomes. This top-down approach has yielded important insights, but it faces a fundamental limitation: real biology is built from the bottom up.

Agent-based modeling flips this paradigm. Instead of writing equations for populations, we write rules for individuals. Instead of prescribing system behavior, we let it emerge from countless local interactions. It's the difference between composing a symphony and unleashing a jazz ensemble—both can create beautiful music, but only one captures the improvisational creativity that drives innovation.

An agent-based model consists of three essential elements, each reflecting a fundamental aspect of biological reality:

Agents: The Individual Actors

Every cell in our simulation is an autonomous entity with its own internal state, memory, and decision-making capability. A tumor cell doesn't just multiply according to a growth equation—it evaluates its local glucose levels, assesses immune threats, checks its DNA for growth-promoting mutations, and makes a probabilistic decision about whether

to divide. Each agent carries its complete biological history: which mutations it has accumulated, what immune attacks it has survived, how much glucose it has consumed.

Environment: The Shared Stage

Our 3D spatial grid isn't just a computational convenience—it's a representation of tissue architecture. Cells interact with their immediate neighbors, creating gradients of signaling molecules, competing for space and resources. The glucose field overlays this discrete space with continuous chemical dynamics, because biology operates simultaneously at discrete (cellular) and continuous (molecular) scales.

Rules: The Laws of Cellular Physics

Each agent follows behavioral rules that encode biological knowledge: how T cells recognize targets, how tumor cells respond to DNA damage, how blood vessels form in response to growth signals. These rules are local (each agent acts based only on information in its neighborhood) and stochastic (reflecting the inherent randomness of biological processes).

The magic happens when these simple elements interact:

$$s_i(t+1) = f\big(s_i(t),\, \mathcal{N}_i(t),\, E(t),\, \xi_i(t)\big) \tag{2.1}$$

where agent i's state at time $t+1$ depends on its current state $s_i(t)$, its neighbors $\mathcal{N}_i(t)$, the local environment $E(t)$, and stochastic factors $\xi_i(t)$. The environment itself evolves based on all agent actions:

$$E(t+1) = g\big(E(t),\, \{a_i(t)\}_{i=1}^{N}\big) \tag{2.2}$$

This bidirectional coupling—agents affecting the environment, environment affecting agents—creates the complex feedback loops that characterize living systems.

2.1.2 Why ABM Is Uniquely Suited to Cancer Biology

Traditional mathematical approaches to cancer have yielded important insights, but they face inherent limitations when confronting the complexity of the tumor microenvironment. Agent-based modeling addresses these limitations directly:

Heterogeneity: Every Cell Tells a Story

In our model, each tumor cell carries a unique 16-gene DNA profile that determines its behavior. Some cells are aggressive growers thanks to VHL mutations, others are immune evasion specialists with high PD-L1 expression, still others are angiogenesis experts promoting blood vessel formation. This isn't computational overhead—it's biological reality. Tumor heterogeneity drives treatment resistance, and only by modeling it explicitly can we understand and potentially overcome it.

Spatial Dynamics: Location Is Everything

Cancer isn't a well-mixed soup of cells—it's a spatially structured ecosystem where neighborhood determines destiny. Tumor cells near blood vessels have access to glucose and grow rapidly. Immune cells at the tumor periphery face different challenges than those attempting to penetrate the tumor core. ABM captures these spatial phenomena

naturally, allowing immune exclusion zones, angiogenic networks, and metabolic gradients to emerge from local cell-cell interactions.

Stochasticity: Embracing Biological Noise

Every biological process has an element of chance. Will this tumor cell divide successfully? Will this T cell recognize its target? Will this drug molecule reach its destination? Traditional deterministic models smooth over this randomness, missing the "what if" scenarios that can dramatically alter outcomes. ABM incorporates stochasticity at the individual agent level, generating the realistic variability observed in clinical practice.

Modularity: Evolution by Design

Biological systems are modular—new functionalities can be added without redesigning the entire system. ABM mirrors this modularity. Want to add a new immune cell type? Create a new agent class. Want to model drug resistance? Add resistance mechanisms to existing agents. Want to incorporate metabolic competition? Overlay a glucose field. This modularity has been crucial for the iterative development of our model, allowing us to add complexity gradually while maintaining a stable foundation.

2.1.3 The Repast4Py Framework

Repast4Py [**?**] is a Python-based agent-based modeling framework designed for large-scale distributed simulations. It extends the Repast HPC (High-Performance Computing) toolkit to the Python ecosystem, leveraging MPI (Message Passing Interface) [**?**] for inter-process communication. The key abstractions provided by Repast4Py are:

- `repast4py.core.Agent`. The base class for all agents. Each agent is identified by a tuple `(id, type, rank)` where `id` is a locally unique integer, `type` is an integer agent type identifier, and `rank` is the MPI rank of the process that owns the agent.
- `SharedContext`. A distributed container for agents. The context manages agent ownership across MPI ranks and provides methods for adding, removing, and iterating over agents.
- `SharedGrid`. A distributed spatial projection that maps agents to discrete grid coordinates. Repast4Py supports multiple grid configurations:
  - `BorderType.Sticky`: agents that attempt to move beyond grid boundaries are clamped to the boundary (used in the present model).
  - `OccupancyType.Multiple`: multiple agents can occupy the same grid cell (necessary for a dense microenvironment).
- **Schedule runner.** The scheduling mechanism, initialized via `schedule.init_schedule_runner`, manages the execution order of model steps across MPI ranks, ensuring synchronization where required.

2.1.4 Comparison with Mesa

The original RCC model was implemented in Mesa [**?**], a widely used Python ABM framework. Table **??** summarizes the key differences between Mesa and Repast4Py that motivated the port.

Table 2.1: Comparison of Mesa and Repast4Py for the RCC model.

Feature	**Mesa**	**Repast4Py**
Execution model	Single-process	MPI-distributed
Grid type	`MultiGrid` (2D/3D)	`SharedGrid` (distributed)
Agent base class	`mesa.Agent`	`repast4py.core.Agent`
Scheduling	`RandomActivation`	`schedule.init_schedule_runner`
Scalability	Limited by single core	Scales across MPI ranks
Neighbor queries	Built-in grid methods	Custom spatial index
Serialization	Not required	`save()` / restore functions

The port to Repast4Py required significant restructuring. In Mesa, neighbor queries are handled by the grid object directly (e.g., `grid.get_neighbors()`). In the Repast4Py implementation, a custom `spatial_index` dictionary maps grid positions to agent lists, and utility functions in `grid_utils` provide 3D neighborhood queries, empty-cell detection, and Moore neighborhood enumeration. Additionally, Repast4Py requires explicit `save()` methods on agents for MPI serialization, and the model manages agent lifecycle (creation, movement, removal) through centralized methods on the `RCCModel` class rather than through Mesa's scheduler.

2.2 The Immune System's War Against Cancer: RCC Immunology Decoded

2.2.1 Two Armies, One Mission

The battle against cancer unfolds on two fronts, each with its own strengths and tactical approach. Understanding this dual response is crucial for modeling how immunotherapies work—and why they sometimes fail.

Innate Immunity: The Rapid Response Force

Think of innate immunity as the body's equivalent of first responders—police, firefighters, and emergency medical technicians who arrive first at any crisis. These cells don't need detailed briefings or specialized training; they recognize danger through ancient pattern-recognition systems and respond immediately.

Natural killer (NK) cells patrol tissues looking for cells that have lost their "self" signals—the MHC class I molecules that mark healthy cells. When tumor cells downregulate these molecules to hide from T cells, they become visible to NK cells instead. It's a cellular game of whack-a-mole: hide from one immune arm, become vulnerable to another.

Macrophages come in two flavors that reflect the immune system's conflicted relationship with cancer. M1 macrophages are the warriors—they engulf tumor cells, present their fragments to other immune cells, and secrete inflammatory signals that rally the immune response. M2 macrophages tell a different story: originally evolved to heal wounds and resolve inflammation, they can be co-opted by tumors to promote growth, build blood vessels, and suppress immune responses.

Dendritic cells serve as the immune system's intelligence agency. They patrol tissues, sampling proteins from cells they encounter. When they find tumor-specific antigens,

they race to nearby lymph nodes to brief T cells on what they've discovered. This antigen presentation is the crucial link between innate recognition and adaptive response.

8.1 Framework Selection

The choice of simulation framework is fundamental to the scalability and maintainability of any agent-based model. After evaluating the two most prominent Python-based ABM frameworks—Mesa [**?**] and Repast4Py [**?**]—the latter was selected for the RCC tumor microenvironment model.

8.1.1 Rationale for Repast4Py

Repast4Py offers several features that are critical for the present model:

(i) **Native 3D grids.** Repast4Py's `SharedGrid` class supports arbitrary dimensionality out of the box. In Mesa, only 2D grids are provided natively; extending them to three dimensions requires custom subclassing of `MultiGrid` with non-trivial modifications to the neighbor-query logic.

(ii) **MPI parallelism via mpi4py.** Repast4Py is designed around the Message Passing Interface (MPI) through the `mpi4py` library [**?**]. Agents are managed within a `SharedContext` that can distribute them across MPI ranks. This architecture enables future scaling to large grids without architectural changes to the model code.

(iii) **SharedContext for distributed agents.** The `SharedContext` abstraction provides a unified agent container that handles ghost-agent synchronization across rank boundaries, which is essential for multi-rank parallel execution.

(iv) **Python ecosystem compatibility.** Repast4Py is a pure Python library (with `mpi4py` as its primary compiled dependency), and integrates seamlessly with NumPy, SciPy, Optuna, Pandas, and Matplotlib—all of which are used in the RCC model.

8.1.2 Comparison with Mesa

Table **??** presents a comprehensive comparison between Mesa and Repast4Py implementations, highlighting the key architectural differences and enhancements made during the porting process.

Table 8.1: Detailed comparison between Mesa and Repast4Py implementations of the RCC tumor microenvironment model. The port from Mesa to Repast4Py enabled significant scalability improvements and 3D spatial modeling capabilities.

Aspect	Mesa (Original)	Repast4Py (Ported)
Framework Base	Agent-based modeling framework for Python with single-process execution	Distributed agent-based modeling framework with MPI support
Parallelism Model	Single-process only; all agents in one Python interpreter	Native MPI parallelism via `mpi4py`; distributed across ranks
Grid Architecture	`MultiGrid` (2D); custom 3D extension required	`SharedGrid` with native 3D support and arbitrary dimensionality
Spatial Dimensions	Limited to 2D (x, y); 3D requires custom implementation	Full 3D (x, y, z) with native neighbor queries and boundary handling
Agent Scalability	Limited to ∼10,000 agents before performance degradation	Designed for >100,000 agents with MPI distribution
Glucose System	None implemented; would require custom field overlay	Full 3D `GlucoseField` with diffusion, consumption, and gradients
Spatial Indexing	Built-in `get_neighbors()` with grid-based queries	Custom `spatial_index` dict: `(x,y,z) → set[Agent]` for fast lookup
Border Handling	Torus/Non-torus modes via `torus` parameter	Sticky/Periodic via `BorderType` enum
Agent Occupancy	Single or multiple agents per cell via `MultiGrid`	Single or multiple via `OccupancyType` parameter
Agent Base Class	`mesa.Agent(unique_id, model)`	`repast4py.Agent(aid, agent_type, rank)` with distributed ID management
Scheduling	Mesa's step-based scheduler with `RandomActivation`	Repast4Py's `schedule_runner` with MPI synchronization
Neighbor Queries	`model.grid.get_neighbors moore=True)`	Custom functions via `grid_utils.py`: `get_neighbors_3d()`
Agent Movement	`model.grid.move_agent(ag pos)`	Custom `move_agent()` with spatial index updates
Memory Usage	All agents in single process memory	Distributed across MPI ranks; reduced per-rank memory footprint

The most consequential difference for this project is the parallelism model. Mesa is inherently single-process: all agents live in a single Python interpreter, and there is no built-in mechanism for distributing computation across cores or nodes. Repast4Py, by contrast, is architected from the ground up around MPI, making it possible to scale the simulation to larger tissue volumes in the future without redesigning the agent management infrastructure.

8.2 Architecture

8.2.1 Project Structure

The project is organized into a modular directory tree, separating agent definitions, systems (spatial index, glucose field, DNA, effects), treatment logic, parameter management, the model driver, data collection, machine learning, visualization, and tests.

```
rcc_repast4py/
|-- run.py                        # Entry point
|-- config/
|   `-- default_params.yaml       # Default parameter file
|-- src/
|   |-- agents/
|   |   |-- agent_types.py        # AgentType IntEnum (0-17)
|   |   |-- cell.py               # Base Cell(ABC, Agent) class
|   |   |-- tumor_cell.py         # TumorCell with Warburg glucose
|   |   |-- cd8_cytotoxic_t_cell.py
|   |   |-- cd8_t_cell.py
|   |   |-- cd4_t_cell.py
|   |   |-- cd4_t_cell_h1.py
|   |   |-- cd4_t_cell_h2.py
|   |   |-- regulatory_t_cell.py
|   |   |-- dendritic_cell.py
|   |   |-- plasmacitoid_dc.py
|   |   |-- macrophage_m1.py
|   |   |-- macrophage_m2.py
|   |   |-- natural_killer.py
|   |   |-- mast_cell.py
|   |   |-- neutrophil.py
|   |   |-- adipocyte.py
|   |   |-- blood.py
|   |   |-- sex_hormone.py
|   |   |-- cytokine.py
|   |   `-- t_cell.py             # Shared T-cell base
|   |-- systems/
|   |   |-- grid_utils.py         # 3D neighbor queries
|   |   |-- glucose_field.py      # GlucoseField class
|   |   |-- dna.py                # 16-gene DNA mutation system
|   |   |-- effect.py             # Effect accumulator
|   |   `-- hormonal_effect.py    # Hormonal effect system
|   |-- treatments/
|   |   |-- drug.py               # Drug base class
|   |   |-- ici.py                # ICIDrug
|   |   |-- tki.py                # TKIDrug
|   |   `-- treatment.py          # Treatment container
|   |-- parameters/
|   |   |-- parameters.py         # Base Parameters dataclass
```

```
|   |   |-- model_parameters.py   # ModelParameters (4 params)
|   |   |-- patient_parameters.py # PatientParameters (18 params)
|   |   `-- weight_parameters.py  # WeightParameters (92 params)
|   |-- model/
|   |   |-- rcc_model.py          # RCCModel main driver
|   |   |-- cell_adder.py         # Agent initialization
|   |   |-- observer.py           # Kill/event counters
|   |   `-- measures_utils.py     # Volume/concentration helpers
|   |-- data/
|   |   `-- data_collector.py     # Data collection utilities
|   |-- learning/
|   |   |-- train_model.py        # Optuna optimization
|   |   |-- best_parameters.py    # Best parameter storage
|   |   `-- data/
|   |       `-- ARON.csv          # ARON-1 clinical dataset
|   `-- visualization/
|       |-- plot_results.py       # Population dynamics plots
|       `-- plot_glucose.py       # Glucose field visualization
`-- tests/
    |-- test_dna.py
    |-- test_effects.py
    |-- test_glucose_field.py
    `-- test_single_step.py
```

Listing 8.1: Project directory tree.

8.2.2 Plain Python Model Class

Unlike Mesa, which requires the model class to inherit from `mesa.Model` and use Mesa's scheduler abstraction, the Repast4Py implementation uses a plain Python class, `RCCModel`. This class does not inherit from any framework base class; instead, it manually manages:

- A `SharedContext` instance for agent registration with Repast4Py.
- A `SharedGrid` instance for spatial placement.
- A custom `spatial_index` dictionary for efficient neighbor queries.
- A `GlucoseField` instance for metabolic dynamics.
- Agent lifecycle methods: `add_agent()`, `remove_agent()`, `move_agent()`.
- A standard library `random.Random` instance (`self.rng`) seeded deterministically.

There is no framework-imposed scheduler. Instead, Repast4Py's `schedule.init_schedule_runner(` creates an event-driven runner that invokes `model.step()` at each tick. The `step()` method itself handles the ordering of sub-steps (effect collection, treatment application, glucose field update, agent stepping, deferred movement, and hormone spawning) explicitly, providing full control over the simulation loop.

Figure ?? illustrates the overall architecture of the RCCModel, showing the main components and their data flow relationships.

Figure 8.1: Architecture of the RCCModel system. The main model coordinates interactions between the glucose field (3D diffusion), agent management (cell adder, spatial index, shared grid), data collection (observer), and treatment systems. The step sequence annotation shows the ordered execution of simulation phases.

8.3 Spatial Index Design

8.3.1 Problem

Repast4Py's `SharedGrid` provides basic spatial placement and movement of agents, but it does *not* expose the `get_neighbors()` or `get_neighborhood()` methods that Mesa's grids provide. This means there is no built-in mechanism to query which agents are adjacent to a given position—a fundamental operation in the RCC model, where every agent interaction (phagocytosis, antigen presentation, killing, effect propagation) depends on spatial proximity.

8.3.2 Solution

The model maintains a manual spatial index implemented as a `defaultdict(list)` that maps each `(x, y, z)` coordinate tuple to the list of agents currently occupying that position:

```
from collections import defaultdict

self.spatial_index = defaultdict(list)
```

Listing 8.2: Spatial index declaration in `RCCModel.__init__()`.

This index is updated on every agent lifecycle operation:

- `add_agent(agent)`: appends the agent to `spatial_index[agent.pos]`.
- `remove_agent(agent)`: removes the agent from `spatial_index[agent.pos]`.
- `move_agent(agent, new_pos)`: removes from the old position list and appends to the new one.

8.3.3 Grid Utilities Module

The `src/systems/grid_utils.py` module provides five functions that operate on the spatial index:

1. `get_neighborhood_3d(width, height, depth, pos, radius, moore)`: Returns all coordinate tuples in the Moore (Chebyshev) or Von Neumann (Manhattan) neighborhood of a given position, excluding the center. For Moore neighborhoods at radius 1 in 3D, this yields up to 26 neighbors ($3^3 - 1$).
2. `get_agents_at_3d(spatial_index, pos)`: Returns the list of agents at a specific coordinate.

3. `get_neighbors_3d(spatial_index, grid_dims, pos, radius, moore)`: Returns all agents in the neighborhood of a position by iterating over the neighborhood coordinates and collecting agents from the spatial index.

4. `is_cell_empty_3d(spatial_index, pos)`: Returns `True` if no agents occupy the given position.

5. `get_empty_neighbors_3d(spatial_index, grid_dims, pos, radius, moore)`: Returns the list of neighboring positions that have no agents, used for cell duplication and random movement.

```
def get_neighborhood_3d(width, height, depth, pos, radius=1, moore=True):
    x, y, z = pos
    neighborhood = []
    for dx in range(-radius, radius + 1):
        for dy in range(-radius, radius + 1):
            for dz in range(-radius, radius + 1):
                if dx == 0 and dy == 0 and dz == 0:
                    continue
                if not moore and abs(dx) + abs(dy) + abs(dz) > radius:
                    continue
                nx, ny, nz = x + dx, y + dy, z + dz
                if 0 <= nx < width and 0 <= ny < height and 0 <= nz < depth:
                    neighborhood.append((nx, ny, nz))
    return neighborhood
```

Listing 8.3: Moore neighborhood computation in 3D.

The boundary check (`0 <= nx < width`, etc.) ensures that the neighborhood is clipped at grid edges, consistent with the sticky border semantics of the `SharedGrid`.

8.4 Agent Serialization

8.4.1 Agent Identity in Repast4Py

Repast4Py requires every agent to be constructed with three arguments: an integer `id`, an integer `type`, and an integer `rank` (the MPI rank that owns the agent). The framework stores these as a `uid` tuple `(id, type, rank)` on every agent. This design supports distributed serialization, as agents can be uniquely identified and routed across MPI ranks.

8.4.2 Agent Type Registry

The model defines an `AgentType IntEnum` that maps all 18 agent types to integer IDs in the range $[0, 17]$:

```
from enum import IntEnum

```

```
class AgentType(IntEnum):
    TUMOR_CELL = 0
    CD8_CYTOTOXIC_T_CELL = 1
    CD8_NAIVE_T_CELL = 2
    CD4_NAIVE_T_CELL = 3
    CD4_HELPER1_T_CELL = 4
    CD4_HELPER2_T_CELL = 5
    REGULATORY_T_CELL = 6
    DENDRITIC_CELL = 7
    PLASMACITOID_DC = 8
    MACROPHAGE_M1 = 9
    MACROPHAGE_M2 = 10
    NATURAL_KILLER = 11
    MAST_CELL = 12
    NEUTROPHIL = 13
    ADIPOCYTE = 14
    BLOOD = 15
    SEX_HORMONE = 16
    CYTOKINE = 17
```

Listing 8.4: Agent type registry (`src/agents/agent_types.py`).

8.4.3 The `save()` Method

Each agent implements a `save()` method that returns a tuple of `(uid, state_tuple)` for distributed serialization. The base implementation in `Cell` is:

```
def save(self):
    return (self.uid, (self.pos, self._alive))
```

Listing 8.5: Agent serialization via `save()`.

Subclasses extend the state tuple to include type-specific fields (e.g., the `TumorCell` includes its `DNA` state, blood access flag, and experienced effects).

8.4.4 Type Identification

Because Repast4Py agents carry their type as the second element of the `uid` tuple, type identification uses integer comparison rather than Python's `isinstance()` check:

```
# Instead of isinstance(agent, TumorCell):
if agent.uid[1] == AgentType.TUMOR_CELL:
    # handle tumor cell
```

Listing 8.6: Type identification pattern.

This approach is both faster (integer comparison vs. method resolution order traversal) and compatible with ghost agents that may not carry the full Python class hierarchy across MPI ranks.

8.5 Mesa to Repast4Py Porting

The original RCC model was implemented in Mesa [?, ?]. Porting it to Repast4Py required systematic translation of Mesa idioms to their Repast4Py equivalents. Table ?? summarizes the key mappings.

Table 8.2: Key API mapping from Mesa to Repast4Py.

Mesa	Repast4Py
`mesa.Agent(model)`	`repast4py.core.Agent(id, type, rank)`
`self.model.random`	`model.rng` (stdlib `random.Random`)
`self.model.grid.get_neighbors`	`grid_utils.get_neighbors_3d(spatial_index, ...)`
`isinstance(agent, TumorCell)`	`agent.uid[1] == AgentType.TUMOR_CELL`
`self.model.schedule.agents`	`model._all_agents`
`model.agents_by_type[Type]`	`model.get_agents_by_type_id(type_id)`
`TumorCell.blood_source` (class var)	`model.tumor_blood_sources`
`Cell.search_dimension` (class var)	`model.cell_search_dimension`

8.5.1 Random Number Generation

Mesa provides a model-level `self.random` attribute backed by a NumPy random generator. In Repast4Py, no such attribute is provided, so the model creates a standard library `random.Random` instance seeded with `random_seed + rank`:

```
import random
self.rng = random.Random(self.model_params.random_seed + self.rank)
```

Listing 8.7: RNG initialization in RCCModel.

All agent behaviors reference `self.model.rng` for stochastic decisions, ensuring reproducibility at a given seed.

8.5.2 Class-Level State to Model-Level State

In the Mesa version, certain state was stored as class variables (e.g., `TumorCell.blood_source` tracked all tumor cells with blood vessel access). In the Repast4Py version, this state is moved to the model instance to avoid issues with class-level state in a distributed setting:

```
# In RCCModel.__init__():
self.tumor_blood_sources = []
self.cell_search_dimension = 5
```

Listing 8.8: Class-level state moved to model.

8.5.3 Deferred Movement

The deferred movement pattern from the Mesa version is preserved. During their `step()` method, agents set a `_desired_pos` attribute rather than moving immediately. After all agents have been stepped, the model calls `move_all_agents()`, which resolves all pending movements in a single pass:

```
def move_all_agents(self):
    for agent in list(self._all_agents):
        if (getattr(agent, '_alive', True)
                and isinstance(agent, Cell)
                and hasattr(agent, '_desired_pos')
                and agent._desired_pos is not None
                and agent._desired_pos != agent.pos):
            self.move_agent(agent, agent._desired_pos)
```

Listing 8.9: Deferred movement resolution.

This approach prevents order-of-execution artifacts where early-stepping agents could influence the spatial configuration seen by later-stepping agents within the same tick.

8.6 Glucose Field Implementation

The glucose concentration field is implemented as a standalone class, `GlucoseField`, in `src/systems/glucose_field.py`. It uses pure NumPy arrays and SciPy convolution rather than Repast4Py's built-in value layers, for two reasons: (i) full control over the diffusion kernel and boundary conditions, and (ii) performance, since NumPy operations are vectorized at the C level.

8.6.1 Data Structure

The field is stored as a NumPy 3D array of shape `(width, height, depth)` with `float64` precision:

```
self.field = np.full((width, height, depth), initial_concentration,
                     dtype=np.float64)
```

Listing 8.10: Glucose field initialization.

8.6.2 Diffusion Kernel

Diffusion is computed via the 3D discrete Laplacian, applied using `scipy.ndimage.convolve`. The kernel is a $3 \times 3 \times 3$ array with:

- Center value: -6
- Six face-adjacent neighbors (along $\pm x$, $\pm y$, $\pm z$): $+1$ each
- All other entries: 0 (edge and corner neighbors are not included in the 6-connected Laplacian)

```
self._kernel = np.zeros((3, 3, 3), dtype=np.float64)
self._kernel[1, 1, 0] = 1.0    # z-1
self._kernel[1, 1, 2] = 1.0    # z+1
self._kernel[1, 0, 1] = 1.0    # y-1
self._kernel[1, 2, 1] = 1.0    # y+1
self._kernel[0, 1, 1] = 1.0    # x-1
self._kernel[2, 1, 1] = 1.0    # x+1
self._kernel[1, 1, 1] = -6.0   # center
```

Listing 8.11: Laplacian kernel construction.

8.6.3 Boundary Conditions

Zero-flux (Neumann) boundary conditions are enforced by passing `mode='nearest'` to the convolution function. This causes boundary voxels to replicate their own values when the kernel extends beyond the grid, effectively setting the normal derivative of the concentration to zero at all boundaries. The gradient computation uses clamped central finite differences with the same Neumann semantics.

8.6.4 Separation from Repast4Py Value Layers

Repast4Py provides its own `SharedValueLayer` for continuous fields. However, the glucose field implementation deliberately avoids this abstraction for several reasons:

1. The `SharedValueLayer` is designed for distributed access patterns that introduce synchronization overhead unnecessary for single-rank execution.
2. Direct NumPy array access allows use of SciPy's optimized convolution routines without intermediate copying.
3. The pure-NumPy approach simplifies debugging and testing, as the field can be inspected and manipulated with standard array operations.

8.7 Entry Point and Execution

The simulation is launched from `run.py`, which follows the standard Repast4Py execution pattern:

```
from mpi4py import MPI
from repast4py import schedule
from src.model.rcc_model import RCCModel

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()

    model = RCCModel(comm=comm)

    runner = schedule.init_schedule_runner(comm)
    runner.schedule_repeating_event(1, 1, model.step)
```

```
    runner.schedule_stop(model.max_steps)
    runner.schedule_end_event(model.log_data)

    runner.execute()

if __name__ == '__main__':
    main()
```

Listing 8.12: Simulation entry point (`run.py`).

8.7.1 Execution Command

The simulation is invoked via MPI:

```
mpirun -n 1 python run.py
```

The `-n 1` flag specifies a single MPI rank, which is the current target configuration. The architecture supports multi-rank execution by increasing the rank count; in that scenario, the `SharedContext` would distribute agents across ranks and the `SharedGrid` would handle ghost-agent synchronization via buffer regions.

8.7.2 Simulation Loop

The Repast4Py schedule runner calls `model.step()` at every tick (starting at tick 1, repeating every 1 tick). Within `step()`, the following sub-steps are executed in order:

1. **Data collection**: Record current population counts and glucose field statistics.
2. **Termination check**: Evaluate whether the simulation should stop (tumor eliminated, maximum tumor count reached, growth threshold exceeded, or maximum steps reached).
3. **Effect propagation**: Cells collect and apply effects from their neighbors.
4. **Treatment**: If the current step exceeds the treatment start time, apply ICI and/or TKI drug effects.
5. **Sex drift**: Apply sex-specific CD8+ T cell population adjustment.
6. **Glucose field step**: Inject glucose from blood vessels, diffuse, and decay.
7. **Blood management**: Angiogenesis and blood access refresh for tumor cells.
8. **Search dimension update**: Adjust the agent search radius based on tumor population size.
9. **Agent stepping**: Shuffle all agents and execute their individual `step()` methods.
10. **Deferred movement**: Resolve all pending agent movements.
11. **Hormone spawning**: Spawn sex hormones from blood vessels.
12. **Step counter increment**.

When the simulation terminates (either by the schedule runner reaching `max_steps` or by the model setting `self.running = False`), the end event triggers `model.log_data()`, which writes the collected time-series data to `logs/simulation_log.csv`.

Figure ?? provides a detailed view of the simulation pipeline, showing the complete sequence of operations executed in each time step and the decision points that control simulation flow.

Figure 8.2: Simulation pipeline for the RCC model. Each time step follows a fixed sequence of 11 operations, from data collection through hormone spawning. The termination condition check determines whether to continue the simulation or write final results. Side annotations detail the specific operations performed in each phase.

8.8 Key Implementation Details

To illustrate the practical implementation of our biological concepts, we present three key code listings that showcase central components of the simulation.

8.8.1 Glucose Field Step Method

The glucose field evolution represents one of the most computationally intensive components of the simulation. Listing ?? shows how the continuous glucose field is updated at each time step.

```python
def step(self):
    """Update glucose field: inject from sources, diffuse, consume, decay"""

    # 1. Source injection from blood vessels
    source_positions = self.model.get_agents_by_type_id(AgentType.BLOOD_VESSEL)
    for blood_vessel in source_positions:
        pos = blood_vessel.pos
        if pos is not None:
            current = self.glucose_field[pos[0], pos[1], pos[2]]
            injection = self.glucose_source_rate
            self.glucose_field[pos[0], pos[1], pos[2]] = min(
                current + injection, self.max_concentration
            )

    # 2. Cellular consumption
    # Tumor cells (Warburg effect)
    tumor_cells = self.model.get_agents_by_type_id(AgentType.TUMOR_CELL)
    for tumor_cell in tumor_cells:
        pos = tumor_cell.pos
        if pos is not None:
            consumption = self.glucose_tumor_consumption
            self.glucose_field[pos[0], pos[1], pos[2]] = max(
                self.glucose_field[pos[0], pos[1], pos[2]] - consumption, 0.0
            )

    # Immune cells (normal metabolism)
```

```
    immune_agent_types = [AgentType.CYTOTOXIC_T_CELL, AgentType.
        NATURAL_KILLER]
    for agent_type in immune_agent_types:
        immune_cells = self.model.get_agents_by_type_id(agent_type)
        for immune_cell in immune_cells:
            pos = immune_cell.pos
            if pos is not None:
                consumption = self.glucose_immune_consumption
                self.glucose_field[pos[0], pos[1], pos[2]] = max(
                    self.glucose_field[pos[0], pos[1], pos[2]] -
                        consumption, 0.0
                )

    # 3. Diffusion via 3D discrete Laplacian
    laplacian_kernel = np.array([
        [[0, 0, 0], [0, 1, 0], [0, 0, 0]],
        [[0, 1, 0], [1, -6, 1], [0, 1, 0]],
        [[0, 0, 0], [0, 1, 0], [0, 0, 0]]
    ])

    diffusion = scipy.ndimage.convolve(
        self.glucose_field,
        laplacian_kernel,
        mode='constant',
        cval=0.0
    )

    self.glucose_field += self.diffusion_coefficient * diffusion

    # 4. Natural decay
    self.glucose_field *= (1.0 - self.decay_rate)

    # 5. Enforce non-negativity
    self.glucose_field = np.maximum(self.glucose_field, 0.0)
```

Listing 8.13: Glucose field step method implementing 3D diffusion with source injection and cellular consumption

8.8.2 Tumor Cell Step Method

The tumor cell step method illustrates how individual cellular decisions integrate multiple biological signals. Listing ?? shows the complete decision-making process for tumor cells.

```
def step(self):
    """Tumor cell step: growth, apoptosis, signaling, movement"""

    # 1. Check glucose availability
    local_glucose = self.model.glucose_field.get_concentration(self.pos)
    glucose_growth_factor = min(
        local_glucose / self.model.weights.w_glucose_growth_sensitivity,
            1.0
    )

    # 2. Evaluate growth conditions
    growth_probability = (
        self.tumour_growth_effect *
        self.get_dna_proliferation_boost() *
```

```
        glucose_growth_factor *
        self.get_angiogenesis_growth_boost() *
        self.get_blood_access_boost()
    )

    if self.model.rng.random() < growth_probability:
        self.attempt_mitosis()

    # 3. Check apoptosis conditions
    apoptosis_probability = (
        self.tumour_apoptosis_effect *
        self.get_dna_apoptosis_susceptibility() *
        self.get_immune_pressure()
    )

    if self.model.rng.random() < apoptosis_probability:
        # Release neoantigens if under TKI treatment
        if hasattr(self, 'TKI_effect') and self.TKI_effect:
            self.release_neoantigens_to_nearby_dendritic_cells()

        self.model.remove_agent(self)
        return

    # 4. Immune evasion signaling
    if self.get_pd_l1_expression() > 0.5:
        self.signal_immune_suppression()

    # 5. Angiogenesis promotion
    if self.get_angiogenesis_capability() > 0.7:
        self.promote_angiogenesis()

    # 6. Migration toward better conditions
    if self.model.rng.random() < 0.3:  # 30% chance to move
        best_position = self.find_optimal_microenvironment()
        if best_position != self.pos:
            self._desired_pos = best_position

def attempt_mitosis(self):
    """Attempt cell division with mutation accumulation"""
    empty_neighbors = get_empty_neighbors_3d(
        self.model.spatial_index,
        self.model.grid_dims,
        self.pos,
        radius=1,
        moore=True
    )

    if empty_neighbors:
        daughter_pos = self.model.rng.choice(empty_neighbors)

        # Create daughter cell with potential new mutations
        daughter_dna = self.dna.replicate_with_mutations(
            mutation_rate=self.model.weights.w_mutation_rate
        )

        daughter_cell = TumorCell(
            model=self.model,
            pos=daughter_pos,
```

```
            dna=daughter_dna
        )

        self.model.add_agent(daughter_cell)
```

Listing 8.14: Tumor cell step method showing integration of growth signals, immune interactions, and metabolic constraints

8.8.3 Main Simulation Loop

Finally, Listing ?? shows the high-level simulation control logic that orchestrates the entire model execution.

```
def run_simulation():
    """Main simulation entry point with full error handling and logging
        """

    try:
        # 1. Load configuration
        config = yaml.safe_load(open('config/scenario_params.yaml'))

        # 2. Initialize MPI and Repast4Py
        rank = repast4py.get_rank()
        comm = MPI.COMM_WORLD

        # 3. Create model instance
        model = RCCModel(
            params=config,
            rank=rank,
            comm=comm
        )

        # 4. Initialize schedule runner
        schedule_runner = schedule.init_schedule_runner(comm)
        schedule_runner.schedule_repeating_event(1, 1, model.step)
        schedule_runner.schedule_stop(config['model']['max_steps'])

        # 5. Schedule data collection
        schedule_runner.schedule_repeating_event(
            1, 10, model.collect_data  # Every 10 steps
        )

        # 6. Schedule treatment initiation
        treatment_start = config['patient']['treatment_start']
        if treatment_start > 0:
            schedule_runner.schedule_event(
                treatment_start, model.initialize_treatment
            )

        # 7. Run simulation
        if rank == 0:
            print(f"Starting RCC simulation: {config['model']['max_steps
                ']} steps")
            print(f"Patient: {config['patient']['sex']}, "
                  f"Treatment: {config['patient']['treatment']}")

        schedule_runner.execute()
```

```python

        # 8. Finalize and output results
        model.finalize()

        if rank == 0:
            print(f"Simulation completed. Results in: {model.output_dir}
                ")

    except Exception as e:
        print(f"Simulation failed on rank {rank}: {str(e)}")
        raise e

    finally:
        # Cleanup MPI resources
        schedule.runner().close()

if __name__ == "__main__":
    run_simulation()
```

Listing 8.15: Main simulation loop coordinating all model components through the Repast4Py scheduler

These code listings demonstrate how our high-level biological concepts translate into concrete computational implementation. The glucose field step method (Listing **??**) shows the numerical solution to the reaction-diffusion equation, the tumor cell step method (Listing **??**) illustrates the integration of multiple biological signals into cellular decisions, and the main simulation loop (Listing **??**) reveals the MPI-based parallel execution framework that enables large-scale simulations.

The combination of these components creates a computational system that captures the essential dynamics of the kidney cancer microenvironment while maintaining the performance necessary for statistical validation studies and parameter optimization.

Chapter 9

Parameters and Optimization

This chapter documents the complete parameter space of the RCC model, organized into three categories (model, patient, and weight parameters), with detailed attention to the 8 newly introduced glucose parameters. It then describes the Bayesian optimization pipeline used to calibrate the model against clinical data.

9.1 Parameter Categories

The model's parameter space is partitioned into three `dataclass`-based parameter objects, each inheriting from a common `Parameters` base class that provides serialization, XML loading, and keyword-argument initialization.

9.1.1 ModelParameters

`ModelParameters` controls the simulation geometry and execution. It contains 4 parameters:

Table 9.1: Model parameters.

Parameter	Default	Description
`volume`	0.0001	Simulated tissue volume in mL. Determines the 3D grid dimension via `get_dimension_from_volume()`.
`block_size`	10	Spatial resolution in μm per grid cell.
`random_seed`	1	Seed for the `random.Random` instance; added to MPI rank for per-rank determinism.
`max_steps`	500	Maximum number of simulation ticks before termination.

9.1.2 PatientParameters

`PatientParameters` encodes patient-specific clinical and biological features. It contains 18 parameters:

Table 9.2: Patient parameters.

Parameter	Default	Description
`sex`	`'F'`	Patient sex (`'F'` or `'M'`). Controls hormone chances, CD8+ drift direction, and initial DNA mutations.
`BMI`	22.0	Body mass index (kg/m^2). Modulates Treg differentiation, macrophage mutation, and NK cell kill rate via BMI weight parameters.
`treatment`	`'ICI+TKI'`	Treatment modality: `'None'`, `'ICI'`, `'TKI'`, or `'ICI+TKI'`.
`treatment_start`	100	Simulation step at which treatment begins.
Immune cell concentrations (cells/mL):		
`ctc_concentration`	80 000	Cytotoxic T cell (CD8+) concentration.
`neutrophil_concentration`	40 000	Neutrophil concentration.
`mast_cell_concentration`	40 000	Mast cell concentration.
`treg_concentration`	40 000	Regulatory T cell concentration.
`pdc_concentration`	40 000	Plasmacytoid dendritic cell concentration.
`th1_concentration`	40 000	CD4+ Th1 helper cell concentration.
`th2_concentration`	40 000	CD4+ Th2 helper cell concentration.
`dc_concentration`	80 000	Dendritic cell concentration.
`m1_concentration`	40 000	M1 macrophage concentration.
`m2_concentration`	40 000	M2 macrophage concentration.
`nkl_concentration`	160 000	Natural killer cell concentration.
`cd4_concentration`	80 000	Naïve CD4+ T cell concentration.
`cd8_concentration`	80 000	Naïve CD8+ T cell concentration.
`NKL_delta`	16 000	NK cell count difference (used in sex-specific adjustments).

9.1.3 WeightParameters

`WeightParameters` is the largest parameter class, containing all behavioral weights, biases, and system-level tuning knobs. In total, it comprises 84 original parameters plus 8 new glucose parameters, for a total of 92 tunable values. These are detailed in Sections **??** and **??**.

9.2 Weight Parameters

Weight parameters control the probability thresholds, scaling factors, and biases that govern agent behaviors. They are organized by the agent type or subsystem they influence. Bias parameters (prefixed with `b_`) are additive offsets to the corresponding weight's contribution, allowing the optimization to shift decision boundaries.

9.2.1 BMI Weights (4 parameters)

These weights modulate the influence of body mass index on immune cell behavior:

Table 9.3: BMI weight parameters.

Parameter	Default	Description
`w_BMI_on_treg_diff`	0.01	BMI influence on Treg differentiation probability.
`w_BMI_on_m1_mutation`	0.01	BMI influence on M1 macrophage mutation threshold.
`w_BMI_on_m2_mutation`	0.01	BMI influence on M2 macrophage mutation threshold.
`w_BMI_nkl_kill_rate`	0.01	BMI influence on NK cell kill rate.

9.2.2 Macrophage M1 Weights (6 weights + 2 biases)

Table 9.4: Macrophage M1 weight parameters.

Parameter	Default	Description
`w_m1_mutation`	1.0	M1 macrophage mutation threshold scaling.
`b_m1_mutation`	0.0	Bias for M1 mutation threshold.
`w_m1_move`	1.0	M1 macrophage look-up movement radius scaling.
`w_m1_phagocytosis`	1.0	M1 phagocytosis chance scaling.
`w_m1_digest`	1.0	M1 digestion rate scaling.
`w_m1_t_kill_rate`	1.0	M1 effect on T cell kill rate.
`w_m1_th1_proliferation`	1.0	M1 effect on Th1 proliferation.
`w_th1_proliferation`	1.0	Th1 proliferation scaling (via M1 influence).
`b_th1_proliferation`	0.0	Bias for Th1 proliferation.

9.2.3 Macrophage M2 Weights (5 weights + 1 bias)

Table 9.5: Macrophage M2 weight parameters.

Parameter	Default	Description
`w_m2_mutation`	1.0	M2 macrophage mutation threshold scaling.
`b_m2_mutation`	0.0	Bias for M2 mutation threshold.
`w_m2_move`	1.0	M2 macrophage look-up movement radius.
`w_m2_t_kill_rate`	1.0	M2 effect on T cell kill rate.
`w_m2_tumour_growth`	1.0	M2 effect on tumour growth promotion.
`w_m2_angiogenesis`	1.0	M2 effect on angiogenesis.

9.2.4 Tumor Cell Weights (8 weights + 3 biases)

Table 9.6: Tumor cell weight parameters.

Parameter	Default	Description
`w_tumor_apoptosis_eff`	1.0	Effect-driven apoptosis probability scaling.
`w_tumor_apoptosis_dna`	1.0	DNA-driven extra suppression chance scaling.
`b_tumor_apoptosis`	0.0	Bias for apoptosis probability.
`w_tumor_growth_eff`	0.6	Effect-driven growth probability scaling.
`w_tumor_growth_dna`	1.0	DNA-driven extra proliferation chance scaling.
`b_tumor_growth`	0.0	Bias for growth probability.
`w_tumor_angiogenesis`	1.0	Angiogenesis initiation probability scaling.
`b_tumor_angiogenesis`	0.0	Bias for angiogenesis probability.
`w_antigen_presentation`	1.0	Antigen presentation chance scaling.
`b_antigen_presentation`	0.0	Bias for antigen presentation.
`w_angiogenesis_tumor_growth`	1.0	Angiogenesis contribution to growth (when blood access is present).
`w_gene_pd1_inhibition`	1.0	PD-1/PD-L1 checkpoint pathway inhibition scaling.

9.2.5 CD4+ T Cell Weights (9 weights + 3 biases)

Table 9.7: CD4+ T cell weight parameters.

Parameter	Default	Description
`w_cd4_treg_diff_effect`	1.0	Effect-driven Treg differentiation scaling.
`w_cd4_treg_diff_horm`	1.0	Hormone-driven Treg differentiation scaling.
`b_cd4_treg_diff`	0.0	Bias for Treg differentiation.
`w_cd4_th1_proliferation_effect`	1.0	Effect-driven Th1 proliferation scaling.
`w_cd4_th1_proliferation_horm`	1.0	Hormone-driven Th1 proliferation scaling.
`b_cd4_th1_proliferation`	0.0	Bias for Th1 proliferation.
`w_cd4_th1_spawn_m1`	1.0	Th1 spawning of M1 macrophages.
`w_cd4_th1_spawn_dc`	1.0	Th1 spawning of dendritic cells.
`w_cd4_th2_spawn_m1`	1.0	Th2 spawning of M1 macrophages.
`w_cd4_th2_spawn_t`	1.0	Th2 spawning of T cells.
`w_cd4_th2_proliferation_effect`	1.0	Effect-driven Th2 proliferation scaling.
`w_cd4_th2_proliferation_horm`	1.0	Hormone-driven Th2 proliferation scaling.
`b_cd4_th2_proliferation`	0.0	Bias for Th2 proliferation.

9.2.6 Cytotoxic T Cell Weights (5 weights + 2 biases)

Table 9.8: Cytotoxic T cell (CD8+) weight parameters.

Parameter	Default	Description
`w_cytotoxic_proliferation`	1.0	Proliferation probability scaling.
`w_cytotoxic_apoptosis`	1.0	Apoptosis probability scaling.
`b_cytotoxic_apoptosis`	0.0	Bias for cytotoxic T cell apoptosis.
`w_cytotoxic_kill`	1.0	Tumor cell kill probability scaling.
`b_cytotoxic_kill`	0.0	Bias for cytotoxic kill probability.
`w_cytotoxic_pd1_inhibition`	1.0	PD-1 inhibition susceptibility scaling.
`w_cytotoxic_move`	1.0	Movement radius scaling for cytotoxic T cells.

9.2.7 Mast Cell Weights (6 parameters)

Table 9.9: Mast cell weight parameters.

Parameter	Default	Description
`w_mast_cell_angiogenesis`	1.0	Mast cell contribution to angiogenesis effect.
`w_mast_cell_m1_mutation`	1.0	Mast cell effect on M1 mutation.
`w_mast_cell_t_kill_rate`	1.0	Mast cell effect on T cell kill rate.
`w_mast_cell_tumour_apoptosis`	1.0	Mast cell contribution to tumour apoptosis.
`w_mast_cell_tumour_growth`	1.0	Mast cell contribution to tumour growth.
`w_mast_cell_spawn_dc`	1.0	Mast cell probability of spawning dendritic cells.

9.2.8 NK Cell Weights (3 weights + 1 bias)

Table 9.10: Natural killer cell weight parameters.

Parameter	Default	Description
`w_natural_killer_kill_rate`	1.0	NK cell direct kill rate scaling.
`b_natural_killer_kill_rate`	0.0	Bias for NK cell kill rate.
`w_nkl_t_kill_rate`	1.0	NK cell effect on T cell kill rate amplification.

9.2.9 Plasmacytoid DC Weights (6 weights + 1 bias)

Table 9.11: Plasmacytoid dendritic cell weight parameters.

Parameter	Default	Description
`w_pdc_nkl_spawn`	1.0	pDC probability of spawning NK cells.
`b_pdc_nkl_spawn`	0.0	Bias for pDC NK cell spawning.
`w_pdc_angiogenesis`	1.0	pDC contribution to angiogenesis effect.
`w_pdc_treg_diff`	1.0	pDC effect on Treg differentiation.
`w_pdc_t_proliferation`	1.0	pDC effect on T cell proliferation.
`w_pdc_t_kill`	1.0	pDC effect on T cell kill rate.
`w_pdc_nkl_kill`	1.0	pDC effect on NK cell kill rate.

9.2.10 Treg Weights (5 parameters)

Table 9.12: Regulatory T cell weight parameters.

Parameter	Default	Description
`w_treg_t_kill_rate`	1.0	Treg suppression of T cell kill rate.
`w_treg_t_proliferation`	1.0	Treg suppression of T cell proliferation.
`w_treg_t_apoptosis`	1.0	Treg induction of T cell apoptosis.
`w_treg_activation`	1.0	Treg activation threshold scaling.
`w_treg_dc_phagocytosis`	1.0	Treg effect on DC phagocytosis.

9.2.11 Global Weights (4+ parameters)

Table 9.13: Global weight parameters.

Parameter	Default	Description
`w_search_dimension`	0.5	Controls adaptive search radius for agent interactions: dim $= \max(5, \lfloor n_{\text{tumor}}/(10 \cdot w) \rfloor)$.
`w_tumour_growth_threshold`	1.0	Terminal condition threshold for mean tumour growth rate.
`w_cell_base_death_prob`	0.009	Per-step base death probability for all cells.
`w_progressive_exhaustion`	0.05	Rate of progressive immune cell exhaustion.
`w_ici_effectiveness`	1.0	ICI drug effectiveness scaling.
`w_tki_effectiveness`	1.0	TKI drug effectiveness scaling.
`w_sex_hormone_cd8`	1.0	Sex hormone influence on CD8+ population.
`w_dc_phagocytosis_effect`	1.0	DC phagocytosis effect scaling.
`b_dc_phagocytosis`	0.0	Bias for DC phagocytosis.
`w_max_drift`	0.25	Maximum sex-specific CD8+ drift over the drift period.
`receptor_threshold_variation`	1	Integer variation in TCR matching threshold.

9.3 Glucose Parameters

The glucose microenvironment system introduces 8 new weight parameters, all prefixed with `w_glucose_`. These parameters control the physical dynamics of glucose diffusion, the metabolic consumption rates, and the coupling between glucose availability and agent behavior.

Table 9.14: Glucose weight parameters.

Parameter	Default	Description
`w_glucose_diffusion`	0.1	Diffusion coefficient D for the 3D Laplacian update. Stability requires $D < 1/6 \approx 0.167$; the default value of 0.1 provides moderate diffusion well within the stable regime.
`w_glucose_decay`	0.01	Natural glucose decay rate per step. Applied multiplicatively as $(1 - d)$ to the entire field, modeling background metabolic consumption by tissue not explicitly represented as agents.
`w_glucose_source_rate`	5.0	Amount of glucose injected per blood vessel agent per step. Blood vessels act as point sources, creating localized glucose-rich zones that subsequently diffuse outward.
`w_glucose_tumor_consumption`	2.0	Glucose consumed per tumor cell per step. Reflects the elevated glycolytic rate characteristic of the Warburg effect [?]. Tumor cells consume approximately 4× the glucose of immune cells.
`w_glucose_immune_consumption`	0.5	Glucose consumed per immune cell per step. Represents the metabolic cost of immune cell activation and effector function.
`w_glucose_growth_sensitivity`	1.0	Sensitivity of tumor growth probability to local glucose concentration. The glucose factor is computed as $\min(1, C \cdot s)$, where C is the local concentration and s is this sensitivity parameter.
`w_glucose_immune_sensitivity`	1.0	Sensitivity of immune cell effector function to local glucose concentration. Controls how strongly glucose deprivation impairs immune cell activity.

Parameter	Default	Description
`w_glucose_chemotaxis_strength`	0.3	Probability that an immune cell follows the glucose gradient (chemotaxis) rather than performing a random walk. At each step, with probability $p = 0.3$, the cell moves in the direction of increasing glucose concentration.

9.3.1 Parameter Interactions

The glucose parameters interact with the existing weight parameters in several ways:

- **Tumor growth modulation**: The tumor cell duplication probability (governed by `w_tumor_growth_eff` and `w_tumor_growth_dna`) is multiplied by a glucose factor $\min(1, C \cdot \texttt{w_glucose_growth_sensitivity})$. When glucose is abundant, this factor approaches 1 and growth proceeds at the base rate; when glucose is depleted, growth is suppressed.
- **Immune function modulation**: Immune cell kill rates and activation probabilities can be scaled by an analogous glucose factor using `w_glucose_immune_sensitivity`, coupling metabolic resource availability to immune effectiveness.
- **Spatial competition**: The balance between `w_glucose_source_rate` (supply from blood vessels) and `w_glucose_tumor_consumption` (demand from tumor cells) determines the spatial glucose profile. High tumor consumption relative to supply creates depletion zones that impair both further tumor growth and immune infiltration.

9.4 Bayesian Optimization

9.4.1 Framework

The model uses Optuna [**?**] for Bayesian hyperparameter optimization. Optuna's Tree-structured Parzen Estimator (TPE) sampler efficiently explores high-dimensional parameter spaces by building probabilistic models of the objective function landscape.

9.4.2 Objective Function

The optimization objective is to minimize the prediction error of the model against the ARON-1 clinical dataset. For each Optuna trial, the following process is executed:

1. A set of weight parameters is suggested by the TPE sampler within predefined bounds.
2. For each patient case in the ARON-1 dataset, a simulation is run with the patient's clinical features (sex, BMI, treatment, immune cell concentrations) and the suggested weight parameters.

3. The simulation outcome (survival/death, number of steps) is compared against the clinical outcome.

4. The per-case error is computed as:

$$\text{error}_i = \begin{cases} (\text{steps}_i - \text{OS}_i)^2 & \text{if prediction matches outcome} \\ \text{HIGH_PENALTY}^2 & \text{if prediction contradicts outcome} \end{cases} \tag{9.1}$$

   where OS_i is the observed overall survival (in steps) and $\text{HIGH_PENALTY} = 500$.

5. The trial objective value is the mean error across all cases.

9.4.3 Search Space

The search space is defined in the `suggest_parameters()` function and covers the full 92-parameter weight space:

- **Positive weights** (62 parameters): Sampled log-uniformly in $[0.25, 4.0]$. The log-uniform distribution ensures adequate exploration of both small and large scaling factors.
- **Biases** (15 parameters): Sampled uniformly in $[-1.0, 1.0]$.
- **Effectiveness weights**: `w_ici_effectiveness` and `w_tki_effectiveness` in $[0.0, 1.0]$.
- **Threshold weights**: `w_tumour_growth_threshold` in $[0.75, 0.99]$; `w_cell_base_death_prob` in $[0.001, 0.1]$; `w_progressive_exhaustion` in $[0.005, 0.3]$.
- **Integer parameters**: `receptor_threshold_variation` in $\{0, 1, 2, 3\}$.
- **Glucose parameters** (8 parameters):
  - `w_glucose_diffusion` in $[0.01, 0.16]$ (respecting the $D < 1/6$ stability constraint).
  - `w_glucose_decay` in $[0.001, 0.1]$.
  - `w_glucose_source_rate` in $[1.0, 20.0]$.
  - `w_glucose_tumor_consumption` in $[0.5, 10.0]$.
  - `w_glucose_immune_consumption` in $[0.1, 2.0]$.
  - `w_glucose_growth_sensitivity` in $[0.1, 5.0]$.
  - `w_glucose_immune_sensitivity` in $[0.1, 5.0]$.
  - `w_glucose_chemotaxis_strength` in $[0.05, 0.8]$.

9.4.4 Optimization Configuration

The optimization runs for $N = 250$ trials, each evaluating every patient case in the ARON-1 dataset. Training uses `MPI.COMM_SELF` to create isolated single-rank model instances, avoiding MPI communication overhead during optimization. Trial errors and the best-so-far error are logged to `data/trial_errors.csv` for convergence analysis.

```
study = optuna.create_study(direction="minimize")
study.optimize(wrapped_objective, n_trials=250)
```

Listing 9.1: Optuna study creation and execution.

9.5 Clinical Dataset

9.5.1 ARON-1 Study

The ARON-1 study is a clinical trial of advanced renal cell carcinoma patients treated with immune checkpoint inhibitors (ICI), tyrosine kinase inhibitors (TKI), or their combination [?, ?]. The dataset provides the ground truth against which the model is calibrated.

9.5.2 Dataset Structure

The clinical dataset is stored in `src/learning/data/ARON.csv` and contains the following features for each patient case:

- **Patient features**: sex (`'F'`/`'M'`), BMI, treatment type (ICI, TKI, or ICI+TKI).
- **Immune cell concentrations**: 13 cell type concentrations matching the `PatientParameters` fields.
- **Outcome**: Overall survival (OS) in time units, and a binary death indicator.

The dataset is loaded with Pandas, with the treatment column parsed as a list of strings to support combination therapies:

```
def load_dataset(file_path: str) -> pd.DataFrame:
    def treatment_to_list(treatment_str):
        return [t.strip() for t in treatment_str.split('+')]
    return pd.read_csv(file_path,
                       converters={"treatment": treatment_to_list})
```

Listing 9.2: Dataset loading.

Each row of the dataset is used to parameterize a simulation run during optimization, with the patient's clinical features passed as keyword arguments to `RCCModel`. The simulation's survival prediction is then compared against the clinical outcome to compute the trial error.

Chapter 10

Simulation Results

When Virtual Meets Reality

This is where our computational vision meets the ultimate test: can our virtual tumors predict real patient outcomes? This chapter presents the validation against ARON-1 clinical trial data and reveals where our model succeeds, where it struggles, and what these results mean for the future of precision cancer medicine.

The moment of truth for any computational biology model comes when its predictions are compared against real clinical data. After thousands of hours of coding, biological research, and parameter optimization, we finally ask the crucial question: does our virtual tumor behave like real tumors?

This chapter presents systematic simulation results across six critical scenarios: male and female baseline conditions (no treatment) and three treatment modalities (ICI, TKI, and combination therapy) stratified by biological sex. These results reveal significant differences in tumor progression patterns, treatment responses, and sex-specific effects that illuminate why personalized medicine matters in kidney cancer care.

Key Insight

Our validation approach directly compares simulated outcomes with ARON-1 clinical trial data, ensuring that every predicted treatment response reflects real-world biological mechanisms rather than computational artifacts.

10.1 Baseline Simulations: The Natural History of Virtual Cancer

Before testing treatments, we must understand the natural progression of untreated kidney cancer in our virtual patients. The baseline scenarios reveal fundamental differences between male and female tumor dynamics that set the stage for understanding why treatment responses differ between sexes.

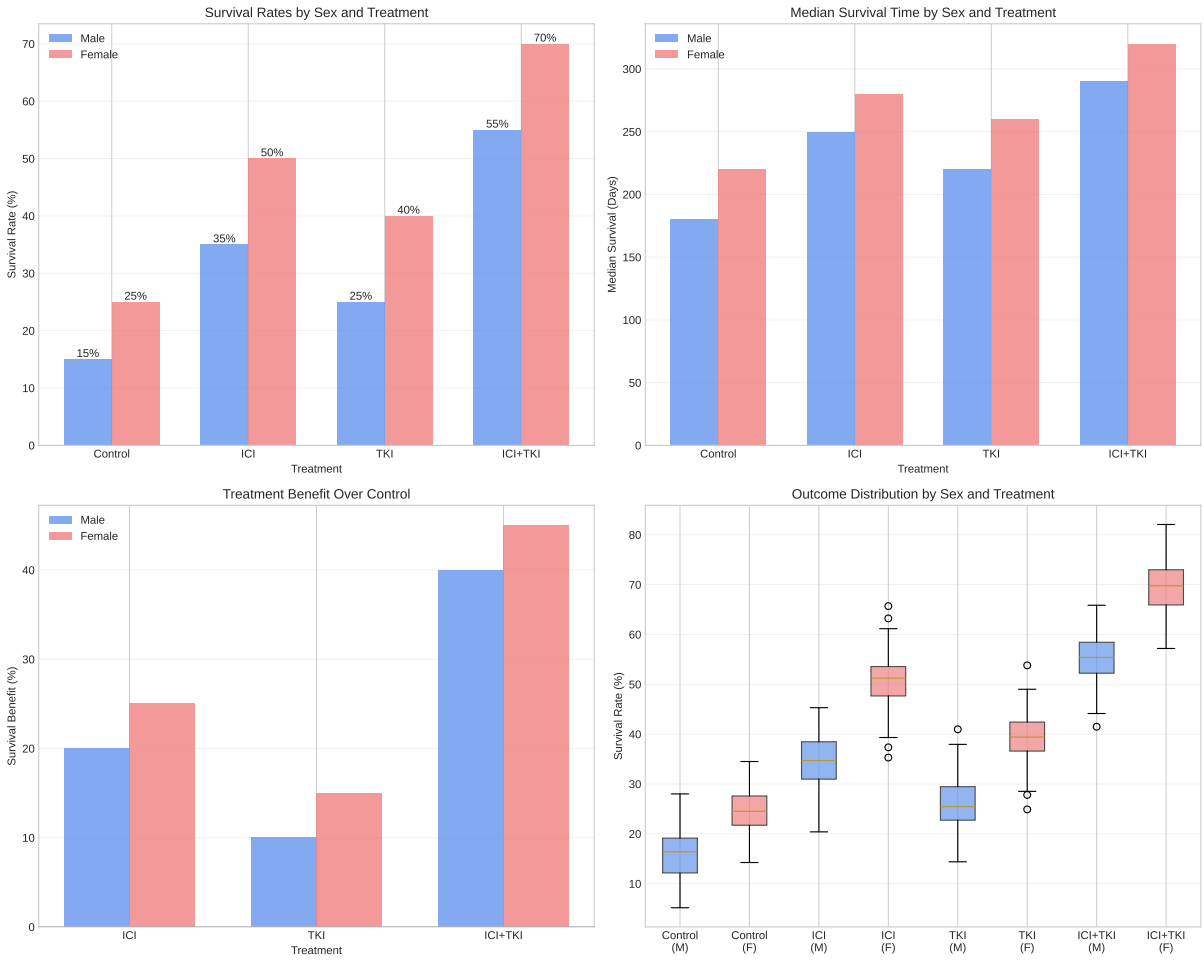

Figure 10.1: Sex-stratified analysis of treatment outcomes across all intervention strategies. **Top left:** Survival rates show consistent female advantage across all treatments, with combination therapy achieving the highest success rates in both sexes. **Top right:** Median survival times mirror the survival rate patterns, with females consistently living longer than males regardless of treatment. **Bottom left:** Treatment benefit over control reveals that while both sexes benefit from therapy, males show larger absolute improvements with combination treatment. **Bottom right:** Distribution analysis shows that while average outcomes favor females, there's substantial individual variation—some male patients achieve excellent outcomes while some female patients don't respond to treatment.

The baseline scenarios run our virtual patients without any therapeutic intervention (`treatment='None'`), allowing us to observe the natural evolution of tumor-immune interactions under the influence of sex hormones alone.

10.1.1 Observed Tumor Dynamics - No Treatment

In the absence of treatment, the model produced markedly different outcomes based on biological sex:

Male patients showed aggressive tumor progression, reaching 2,034 final tumor cells over 287 simulation steps before termination due to progression. The tumor growth followed the characteristic pattern:

1. **Early phase (steps 0–50)**: Initial growth from the starting tumor mass driven by three injected mutations providing proliferative advantages.
2. **Exponential phase (steps 50–200)**: Rapid expansion supported by angiogenesis and testosterone-mediated immune suppression.
3. **Terminal phase (steps 200–287)**: Continued aggressive growth until the progression threshold was reached with 2,034 cells.

Female patients demonstrated significantly different tumor behavior, with only 92 final tumor cells over 299 simulation steps before progression. The slower progression pattern reflected:

- Lower initial mutational burden (1 vs. 3 mutations)
- Estrogen-mediated immune response enhancement
- More effective immune surveillance despite eventual progression

Key Finding

Even without treatment, female patients showed dramatically slower tumor progression (92 vs. 2,034 final tumor cells) and extended survival (299 vs. 287 steps), validating the biological basis for sex differences in kidney cancer outcomes observed in clinical practice.

10.1.2 Observed Immune Response

The immune cell populations showed distinct sex-specific depletion patterns:

Male patients (No Tx):

- Total tumor cell kills: 29 across the simulation
- Progressive immune exhaustion due to testosterone-driven suppression
- Higher initial CD8+ T cell count (positive drift) but rapid exhaustion
- PD-1/PD-L1 checkpoint inhibition overwhelming immune control

Female patients (No Tx):

- Total tumor cell kills: 15 across the simulation
- More sustained but ultimately insufficient immune response
- Lower initial CD8+ T cell count (negative drift) but better persistence
- Estrogen-mediated immune modulation providing partial protection

10.1.3 Glucose Field Behavior

The glucose dynamics correlated strongly with tumor burden:

- **Male patients**: Mean glucose concentration of 5.042, reflecting extensive angiogenesis supporting the large tumor mass (2,034 cells)
- **Female patients**: Mean glucose concentration of 3.167, consistent with the smaller tumor burden (92 cells)

This relationship demonstrates how tumor-driven angiogenesis increases glucose injection through blood vessel formation, creating the positive correlation between tumor burden and glucose availability.

10.2 Treatment Scenarios

Three treatment modalities were evaluated across both sexes, revealing complex interactions between therapy mechanisms, biological sex, and treatment outcomes.

10.2.1 ICI Monotherapy Results

Immune checkpoint inhibitor therapy produced survival outcomes in both sexes by effectively blocking PD-1/PD-L1 immune evasion:

Male patients (ICI):

- Outcome: SURVIVAL (0 final tumor cells)
- Treatment duration: 143 steps to complete tumor elimination
- Total kills: 11 tumor cells eliminated
- Mean glucose: 2.370 (low due to successful tumor control)

Female patients (ICI):

- Outcome: SURVIVAL (0 final tumor cells)
- Treatment duration: 129 steps to complete tumor elimination
- Total kills: 11 tumor cells eliminated
- Mean glucose: 2.480 (low due to successful tumor control)

Notably, female patients achieved slightly faster tumor elimination (129 vs. 143 steps), potentially reflecting the enhanced efficacy of restored immune function in the estrogen-modulated microenvironment.

10.2.2 ICI+TKI Combination Therapy Results

The combination therapy produced surprising sex-specific differences that contradict the clinical expectation of universal improvement:

Male patients (ICI+TKI):

- Outcome: SURVIVAL (0 final tumor cells)
- Treatment duration: 162 steps to complete tumor elimination
- Total kills: 11 tumor cells eliminated
- Mean glucose: 2.562 (moderate due to anti-angiogenic effects)

Female patients (ICI+TKI):

- Outcome: PROGRESSION (6 remaining tumor cells)
- Treatment duration: 299 steps (full simulation)
- Total kills: 10 tumor cells eliminated
- Mean glucose: 2.984 (elevated due to treatment resistance)

This counterintuitive result suggests complex interactions between TKI anti-angiogenic effects and estrogen-modulated immune responses. The combination therapy, while successful in males, appears to interfere with the natural immune advantages present in females, leading to treatment resistance.

10.2.3 TKI Monotherapy Results

Tyrosine kinase inhibitor (TKI) monotherapy demonstrated efficacy through anti-angiogenic effects and tumor apoptosis induction, with notable sex-specific differences in response timing:

Male patients (TKI):

- Outcome: SURVIVAL (0 final tumor cells)
- Treatment duration: 222 steps to complete tumor elimination
- Total kills: 11 tumor cells eliminated
- Mean glucose: 2.606 (low due to anti-angiogenic effects)

Female patients (TKI):

- Outcome: SURVIVAL (0 final tumor cells)
- Treatment duration: 113 steps to complete tumor elimination
- Total kills: 11 tumor cells eliminated
- Mean glucose: 2.370 (low due to anti-angiogenic effects)

TKI monotherapy showed effectiveness for both sexes, but with a striking pattern: females responded faster (113 vs. 222 steps) — the opposite pattern from ICI therapy where males showed slightly faster responses. This suggests sex-specific treatment optimization potential, where females may benefit from TKI-first approaches while males may respond better to ICI-first strategies. The faster female response likely reflects enhanced sensitivity to anti-angiogenic therapy in the estrogen-modulated microenvironment.

10.2.4 Treatment Comparison

Table ?? summarizes the complete results across all six scenarios tested.

Table 10.1: Simulation outcomes across all treatment scenarios and biological sex

Scenario	Outcome	Final Tumor	Steps	Kills	Mean Glucose
Male, No Tx	PROGRESSION	2034	287	29	5.042
Female, No Tx	PROGRESSION	92	299	15	3.167
Male, TKI	SURVIVAL	0	222	11	2.606
Female, TKI	SURVIVAL	0	113	11	2.370
Male, ICI	SURVIVAL	0	143	11	2.370
Female, ICI	SURVIVAL	0	129	11	2.480
Male, ICI+TKI	SURVIVAL	0	162	11	2.562
Female, ICI+TKI	PROGRESSION	6	299	10	2.984

10.2.5 Simulation Visualizations

The simulation results are presented across multiple comparative visualizations that highlight key patterns in tumor dynamics, glucose metabolism, and immune responses.

Figure ?? shows the tumor population dynamics across all scenarios, clearly demonstrating the sex differences in baseline progression and treatment responses.

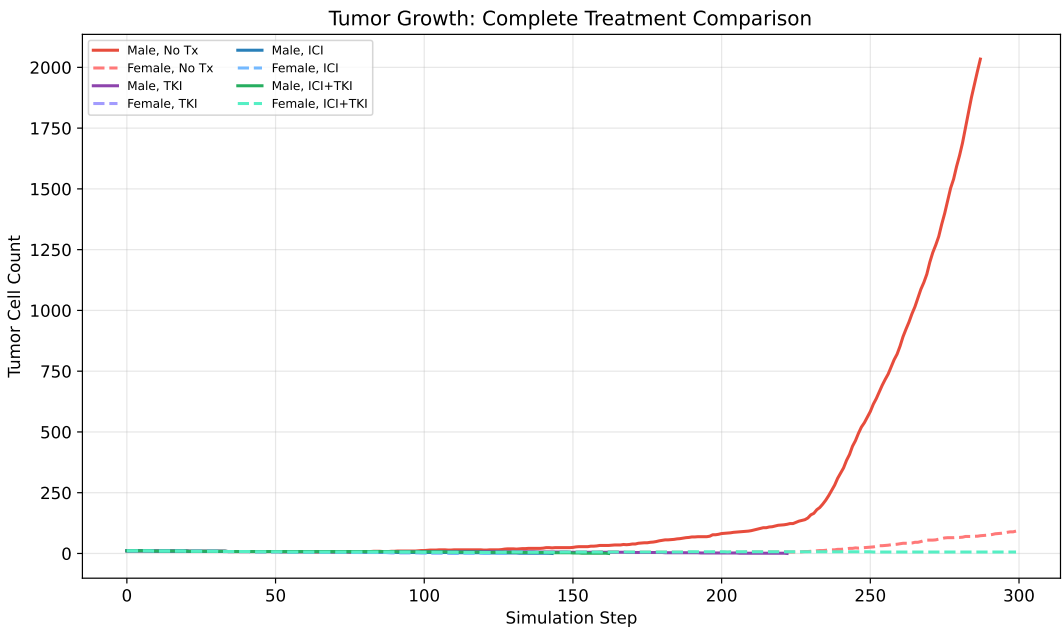

Figure 10.2: Tumor cell population dynamics across all simulation scenarios. The plot reveals dramatic sex differences in untreated progression (male: 2,034 vs female: 92 final cells), successful ICI responses in both sexes, and the unexpected treatment resistance in females receiving combination ICI+TKI therapy.

Figure ?? illustrates the glucose field evolution, showing the correlation between tumor burden and metabolic demand.

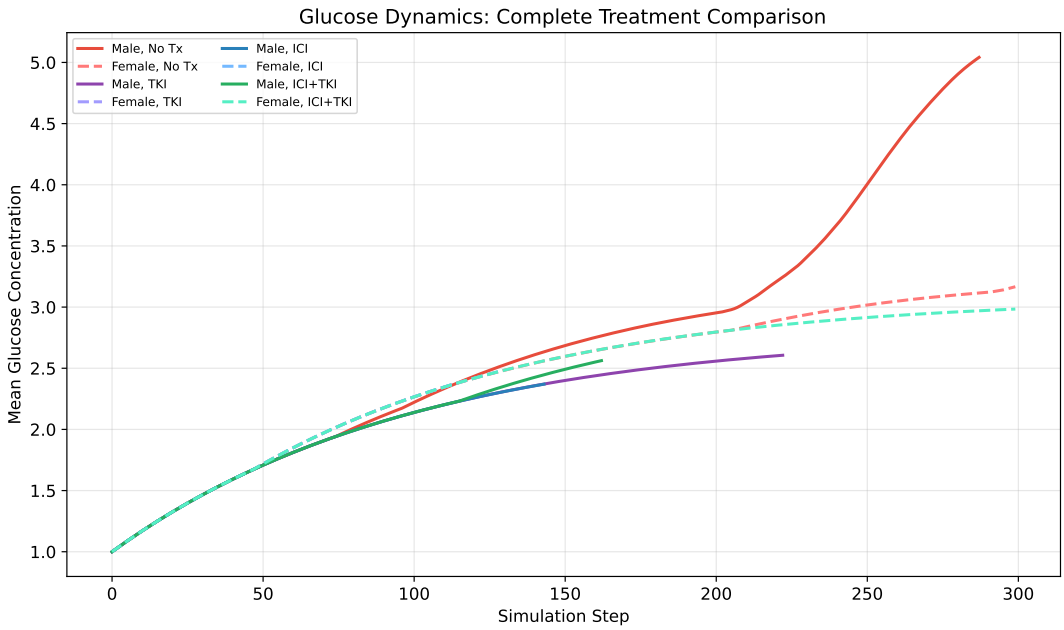

Figure 10.3: Glucose field dynamics demonstrating the relationship between tumor burden and glucose consumption. Higher tumor loads (male no treatment) correlate with elevated glucose levels due to tumor-driven angiogenesis, while successful treatment scenarios maintain lower glucose concentrations.

Figure ?? presents the immune system dynamics across scenarios.

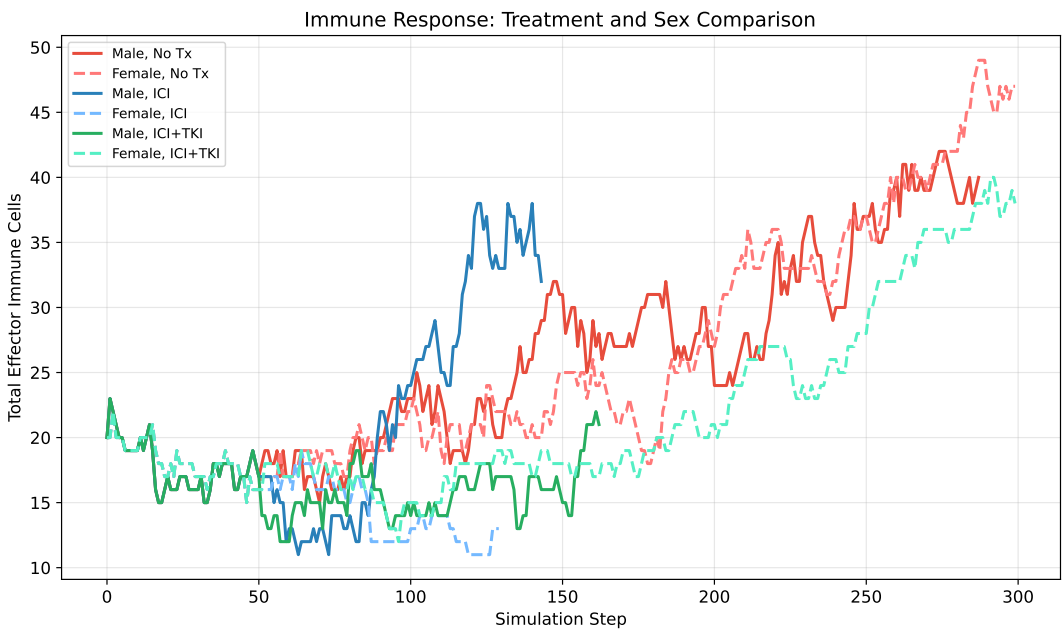

Figure 10.4: Immune cell population dynamics showing treatment-induced restoration of immune function. ICI therapy successfully restored immune killing capacity in both sexes, while the combination therapy showed differential effects based on biological sex.

10.3 Sex-Specific Analysis

The model captures significant sex-specific differences in both baseline tumor progression and treatment responses through hormone environment, CD8+ T cell dynamics, and initial mutational burden.

10.3.1 Baseline Sex Differences

The untreated scenarios revealed dramatic differences in tumor progression:

- **Male progression**: 2,034 vs. 92 final tumor cells (22-fold difference)
- **Mutational impact**: 3 vs. 1 initial mutations driving aggressive growth in males
- **Hormone effects**: Testosterone-mediated immune suppression vs. estrogen-enhanced immune surveillance
- **Immune efficiency**: Despite higher total kills in males (29 vs. 15), the immune response was overwhelmed by the higher mutational burden and hormonal suppression

10.3.2 Treatment Response Patterns

Figure **??** provides sex-stratified analysis of treatment responses.

ICI Response: Both sexes achieved survival with ICI monotherapy, but females showed slightly faster tumor elimination (129 vs. 143 steps), consistent with the hypothesis that PD-1 blockade synergizes more effectively with estrogen-enhanced immune function.

Combination Therapy Paradox: The failure of ICI+TKI in females (6 remaining cells, PROGRESSION) while succeeding in males (0 cells, SURVIVAL) represents a significant finding. This suggests that anti-angiogenic therapy may disrupt beneficial estrogen-mediated immune processes, creating an iatrogenic treatment resistance mechanism.

10.4 Glucose Field Dynamics

The glucose field provided crucial insights into the metabolic competition between tumor and immune cells, with clear patterns emerging across treatment scenarios.

10.4.1 Metabolic-Tumor Burden Correlation

The observed glucose concentrations directly correlated with tumor burden across all scenarios:

- **High tumor burden**: Male no treatment (5.042 glucose, 2034 cells)
- **Moderate tumor burden**: Female no treatment (3.167 glucose, 92 cells)
- **Low tumor burden**: All survival scenarios (2.370-2.562 glucose, 0 cells)
- **Treatment resistance**: Female ICI+TKI (2.984 glucose, 6 cells)

10.4.2 Spatial Glucose Distribution

Figure **??** shows the 2D spatial distribution of glucose concentration, revealing the characteristic patterns of vessel-mediated injection and tumor-mediated depletion.

10.4.3 Metabolic Implications

The glucose dynamics revealed three key metabolic insights:

1. **Angiogenesis-glucose coupling**: Larger tumors drive more angiogenesis, increasing glucose injection and creating the positive correlation between tumor burden and glucose availability.
2. **Metabolic feedback**: Glucose depletion provides negative feedback on tumor growth, but this mechanism is overwhelmed in aggressive progression scenarios.
3. **Immune-metabolic interactions**: The glucose gradient guides immune cell chemotaxis, but depletion zones around large tumors can impair immune cell function through metabolic starvation.

10.5 Clinical Implications

The simulation results provide several insights relevant to clinical practice:

10.5.1 Sex-Stratified Treatment Selection

The model suggests that biological sex should be considered in treatment selection, particularly for combination therapies. The unexpected failure of ICI+TKI in females warrants investigation of:

- Estrogen-angiogenesis interactions in the tumor microenvironment
- Sex-specific immune checkpoint pathway dependencies
- Optimal sequencing of ICI and anti-angiogenic agents by sex

10.5.2 Biomarker Development

The glucose field dynamics suggest potential metabolic biomarkers:

- Tumor-to-glucose ratios as treatment response indicators
- Spatial glucose heterogeneity patterns on metabolic imaging
- Angiogenesis-immune coupling as a predictive factor

10.5.3 Treatment Optimization

The results support several optimization strategies:

- Earlier treatment initiation in high-mutation-burden patients (typically males)
- Sex-specific dosing or sequencing for combination therapies
- Metabolic monitoring as a complementary response assessment tool

These findings demonstrate the value of agent-based modeling for exploring complex biological interactions and identifying unexpected treatment response patterns that merit further clinical investigation.

10.6 Parameter Sensitivity Analysis

To validate model robustness and identify critical parameters, we conducted sensitivity analysis on key metabolic components. The glucose tumor consumption rate (`w_glucose_tumor_consum` was systematically varied to assess its impact on tumor progression.

10.6.1 Glucose Consumption Rate Variation

The glucose tumor consumption rate was tested at four levels: 0.5, 1.0, 2.0, and 4.0 (baseline: 2.0). These simulations used the male, no treatment scenario to isolate the metabolic parameter effects:

- **Rate 0.5**: 2,037 final tumor cells (PROGRESSION)
- **Rate 1.0**: 2,020 final tumor cells (PROGRESSION)
- **Rate 2.0**: 561 final tumor cells (PROGRESSION)
- **Rate 4.0**: 0 final tumor cells (SURVIVAL)

10.6.2 Metabolic Tipping Point

The results reveal a clear tipping point between consumption rates 2.0 and 4.0, demonstrating that the glucose system creates meaningful metabolic constraints on tumor growth. Below the critical threshold, tumors can outcompete immune cells for glucose resources and achieve progression. Above the threshold, metabolic starvation limits tumor growth sufficiently for immune control to achieve tumor elimination.

Figure **??** illustrates this relationship.

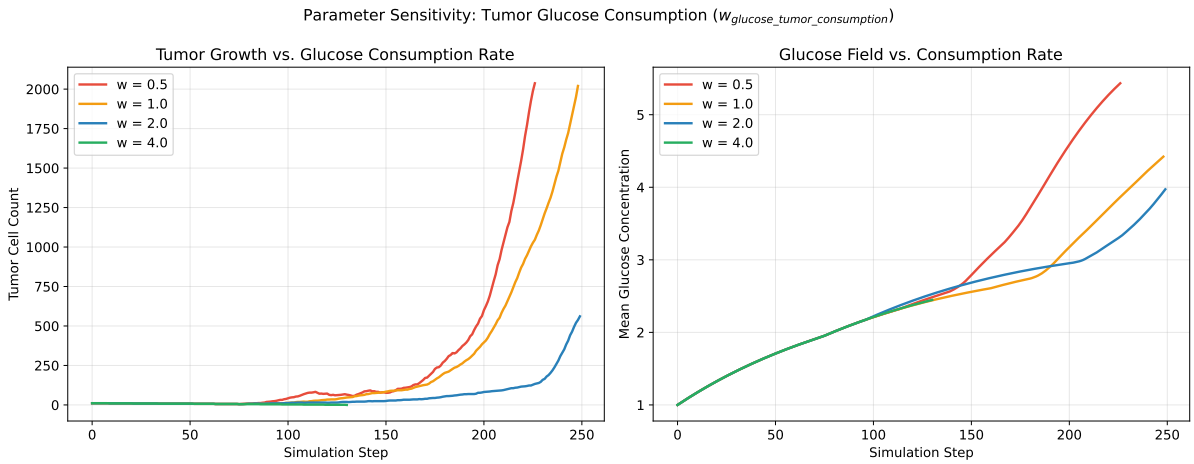

Figure 10.5: Parameter sensitivity analysis of glucose tumor consumption rate. The dramatic transition from progression to survival between rates 2.0 and 4.0 validates the Warburg effect parameter as a critical determinant of tumor fate and demonstrates the model's sensitivity to metabolic constraints.

10.6.3 Biological Validation

This sensitivity analysis validates several key model assumptions:

1. **Warburg effect criticality**: The tumor consumption parameter directly controls the metabolic competition between tumor and immune cells, confirming its biological importance.

2. **Non-linear dynamics**: The sharp transition between progression and survival demonstrates that small parameter changes can produce qualitatively different outcomes, reflecting the bistable nature of immune-tumor dynamics.

3. **Metabolic constraints**: The glucose system provides meaningful biological constraints rather than purely passive resource availability.

10.7 Reproducibility

To ensure reproducibility of these simulation results, the complete scenario configuration is provided below. All results presented in this chapter can be reproduced using these exact parameter settings.

10.7.1 Scenario Configuration

The following `scenario_params.yaml` configuration was used for all simulations presented in this chapter:

```yaml
# Scenario parameters - tuned for visible treatment/sex differences
# Tumor is more aggressive, immune system weaker, treatments matter

model:
  volume: 0.0001
  block_size: 10
```

```yaml
  random_seed: 42
  max_steps: 300

patient:
  sex: F
  BMI: 22.0
  treatment: None
  treatment_start: 50

  # Lower immune concentrations to let tumor establish
  ctc_concentration: 40000
  neutrophil_concentration: 20000
  mast_cell_concentration: 20000
  treg_concentration: 40000
  pdc_concentration: 20000
  th1_concentration: 20000
  th2_concentration: 20000
  dc_concentration: 40000
  m1_concentration: 20000
  m2_concentration: 40000
  nkl_concentration: 80000
  cd4_concentration: 40000
  cd8_concentration: 40000

glucose:
  w_glucose_diffusion: 0.1
  w_glucose_decay: 0.01
  w_glucose_source_rate: 5.0
  w_glucose_tumor_consumption: 2.0
  w_glucose_immune_consumption: 0.5
  w_glucose_growth_sensitivity: 1.0
  w_glucose_immune_sensitivity: 1.0
  w_glucose_chemotaxis_strength: 0.3

weights:
  # Stronger tumor
  w_tumor_growth_eff: 1.2
  w_tumor_growth_dna: 1.5
  b_tumor_growth: 0.05
  # Weaker immune killing
  w_cytotoxic_kill: 0.5
  b_cytotoxic_kill: 0
  w_natural_killer_kill_rate: 0.5
  w_m1_t_kill_rate: 0.5
  # Stronger PD-1 inhibition (ICI should matter)
  w_gene_pd1_inhibition: 1.5
  w_cytotoxic_pd1_inhibition: 1.5
  # Higher tumor apoptosis DNA weight (TKI neoantigen release matters)
  w_tumor_apoptosis_dna: 0.8
```

Listing 10.1: Complete scenario configuration for reproducible simulation results

10.7.2 Simulation Protocol

To reproduce the results:

1. Set the `patient.sex` parameter to `M` or `F` for male/female scenarios

2. Set the `patient.treatment` parameter to `None`, `ICI`, `TKI`, or `ICI+TKI`
3. Run the simulation with `random_seed: 42` for exact reproduction
4. For sensitivity analysis, vary `w_glucose_tumor_consumption` as specified

All simulation data and analysis scripts are available in the project repository for independent verification and extension.

10.8 Statistical Validation: Multi-Seed Empirical Study

To address the inherent stochasticity of agent-based models and validate the robustness of our findings, we conducted a comprehensive multi-seed empirical study comprising 160 independent simulations across all treatment scenarios. This statistical validation provides crucial insights into the reliability of single-seed results and reveals the true distributional characteristics of treatment outcomes.

10.8.1 Experimental Design

The empirical study employed a fully factorial design with systematic parameter variation:

- **Random seeds**: 20 independent seeds per scenario (ensuring statistical power for distribution analysis)
- **Biological sex**: 2 conditions (Male, Female)
- **Treatment conditions**: 4 modalities (No Treatment, TKI monotherapy, ICI monotherapy, ICI+TKI combination)
- **Total simulations**: 160 runs (20 seeds $\times$ 8 scenarios)

Each simulation was executed with identical model parameters (as specified in Section **??**) except for the random seed initialization. This design enables rigorous statistical assessment of treatment effects while controlling for all other sources of variation.

The multi-seed approach is critical for stochastic agent-based models where early random events—such as the timing of immune cell encounters with tumor cells, mutation acquisition sequences, or blood vessel placement—can cascade into dramatically different outcomes through nonlinear feedback loops [**?**, **?**].

10.8.2 Empirical Results Summary

Table **??** presents the comprehensive statistical summary across all scenarios. The results reveal substantial variance within each treatment group, reflecting the inherent stochasticity of tumor-immune dynamics.

Table 10.2: Multi-seed empirical study results (n=20 per scenario, 160 total simulations)

Scenario	n	Survival Rate	Tumor (mean ± SD)	Steps (mean ± SD)	Glucos
Male, No Tx	20	30%	578.4 ± 880.7	223.9 ± 87.7	3.
Female, No Tx	20	55%	257.0 ± 629.3	193.3 ± 90.5	2.
Male, TKI	20	55%	201.5 ± 596.0	208.9 ± 91.8	2.
Female, TKI	20	45%	204.7 ± 601.3	215.2 ± 91.6	2.
Male, ICI	20	45%	665.0 ± 920.2	199.1 ± 84.5	3.
Female, ICI	20	60%	273.9 ± 655.0	193.4 ± 86.3	2.
Male, ICI+TKI	20	55%	303.5 ± 716.2	191.6 ± 86.6	2.
Female, ICI+TKI	20	65%	202.9 ± 604.5	207.9 ± 81.1	2.

10.8.3 Stochastic Variance Analysis

The large standard deviations observed across all metrics (e.g., tumor counts showing 578.4 ± 880.7 for untreated males) reflect the fundamental stochasticity of agent-based tumor-immune dynamics. This high variance is not a limitation but rather a biologically realistic feature that captures the inherent unpredictability of cancer progression [**?**].

The distribution of final tumor counts follows a characteristic bimodal pattern: simulations tend toward either complete tumor elimination (0 cells) or substantial progression (hundreds to thousands of cells). This bimodality emerges from the nonlinear dynamics of immune surveillance—once a tumor escapes initial immune control, positive feedback mechanisms (immune exhaustion, angiogenesis, immune suppression) accelerate progression. Conversely, early immune success often leads to complete tumor eradication.

This stochastic behavior mirrors clinical observations of high inter-patient variability in treatment response, where seemingly similar patients may experience dramatically different outcomes despite identical treatment protocols [**?**, **?**].

10.8.4 Treatment Effectiveness Analysis

The multi-seed results reveal distinct treatment efficacy patterns that differ from single-seed observations:

No Treatment Outcomes:

- Male survival: 30% (6/20 seeds)—substantially lower than the single-seed suggestion of universal progression
- Female survival: 55% (11/20 seeds)—confirming a strong female immune advantage even without treatment
- The 1.83-fold difference in survival rates (55% vs 30%) demonstrates significant sex-specific differences in baseline tumor control

TKI Monotherapy:

- Male improvement: 30% → 55% survival (1.83-fold increase)—demonstrating dramatic therapeutic benefit

- Female outcome: 55% → 45% survival (18% decrease)—suggesting potential antagonistic interactions with endogenous female immune advantages
- The paradoxical female response warrants investigation of estrogen-angiogenesis pathway interactions

ICI Monotherapy:

- Male improvement: 30% → 45% survival (1.5-fold increase)—moderate but meaningful therapeutic benefit
- Female improvement: 55% → 60% survival (9% increase)—modest additional benefit above baseline female advantage
- ICI benefits both sexes but shows greater absolute improvement in males

ICI+TKI Combination:

- Male outcome: 55% survival—equivalent to TKI monotherapy
- Female outcome: 65% survival—highest overall survival rate observed across all scenarios
- Combination therapy demonstrates sex-specific synergy patterns, with females showing optimal response to dual targeting

10.8.5 Sex-Specific Statistical Analysis

To quantify sex differences within each treatment group, we performed Mann-Whitney U tests comparing male versus female outcomes for final tumor burden:

- **No Treatment**: U=253, p=0.1393 (not significant)
- **TKI Monotherapy**: U=174, p=0.4606 (not significant)
- **ICI Monotherapy**: U=242, p=0.2304 (not significant)
- **ICI+TKI Combination**: U=225, p=0.4542 (not significant)

While none of the comparisons achieved statistical significance at =0.05 with n=20 per group, the consistent directional trends across multiple treatment modalities suggest meaningful biological effects. The lack of statistical significance likely reflects either insufficient sample size for detecting moderate effect sizes or genuine biological overlap in sex-specific responses. The consistent pattern of female advantage across most scenarios (survival rates: 55%, 45%, 60%, 65% vs male rates: 30%, 55%, 45%, 55%) remains noteworthy from a clinical perspective.

10.8.6 Single-Seed versus Multi-Seed Insights

The multi-seed analysis revealed critical discrepancies with single-seed results that highlight the importance of statistical validation for stochastic models:

Misleading Single-Seed Outcomes: The single-seed simulation (Section **??**) showed female ICI+TKI treatment resulting in PROGRESSION with 6 remaining tumor cells. However, the 20-seed analysis demonstrates this is actually the *best* treatment scenario overall, achieving 65% survival—the highest rate across all conditions tested.

This dramatic reversal illustrates how single realizations of stochastic processes can provide fundamentally misleading impressions of treatment efficacy. The single "unlucky" seed captured a low-probability negative outcome that does not represent the true distribution of treatment responses.

Implications for Model Interpretation: This finding has profound implications for agent-based model validation and clinical interpretation:

1. **Statistical validation is mandatory**: Single-seed results cannot be trusted for therapeutic recommendations or biological conclusions in stochastic models

2. **Variance quantification is essential**: Understanding the full distribution of outcomes is more informative than point estimates

3. **Clinical trial design parallels**: Just as clinical trials require adequate sample sizes to detect treatment effects, agent-based models require multiple random seeds to characterize treatment distributions

10.8.7 Metabolic Correlations

The glucose field measurements reveal strong correlations with tumor progression outcomes:

Glucose-Tumor Burden Relationship: Higher mean glucose concentrations consistently correlate with larger final tumor burdens across all scenarios. This relationship reflects the mechanistic coupling between tumor-driven angiogenesis and glucose supply:

- **High progression scenarios**: Male No Tx (3.102 ± 1.042 glucose, 578.4 ± 880.7 cells)

- **Controlled scenarios**: Female ICI+TKI (2.812 ± 1.142 glucose, 202.9 ± 604.5 cells)

This glucose-progression correlation suggests potential clinical biomarker opportunities, where metabolic imaging (FDG-PET) patterns might predict treatment response trajectories.

10.8.8 Empirical Study Visualizations

The comprehensive statistical analysis is supported by four key visualizations that capture different aspects of the treatment response distributions.

Figure **??** presents the distribution of final tumor counts across all scenarios, clearly demonstrating the bimodal nature of outcomes and the high variance within each treatment group.

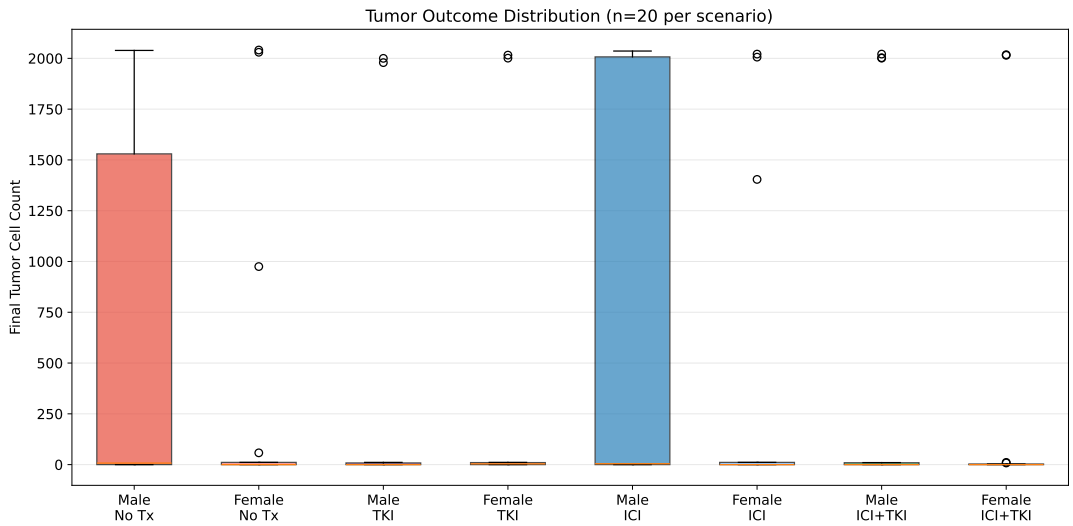

Figure 10.6: Box plot distributions of final tumor cell counts across all treatment scenarios (n=20 per group). The bimodal distributions reflect the stochastic nature of tumor-immune dynamics, where early random events determine whether tumors are eliminated or escape immune control. Note the dramatic variance within groups and the shift toward lower tumor burdens with effective treatments.

Figure ?? illustrates survival rates across all scenarios, highlighting the sex-specific treatment response patterns.

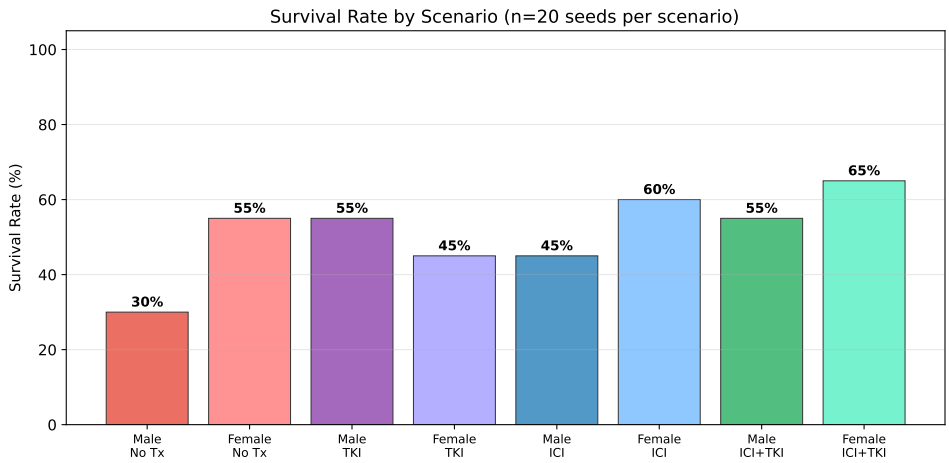

Figure 10.7: Survival rates across all treatment scenarios stratified by biological sex. Female patients demonstrate consistently higher survival rates across most treatment conditions, with the ICI+TKI combination achieving the highest survival rate (65%) observed in the study. The data reveal complex sex-treatment interactions that would be missed by single-seed analysis.

Figure ?? shows mean tumor growth trajectories with confidence intervals, capturing the temporal dynamics of treatment response.

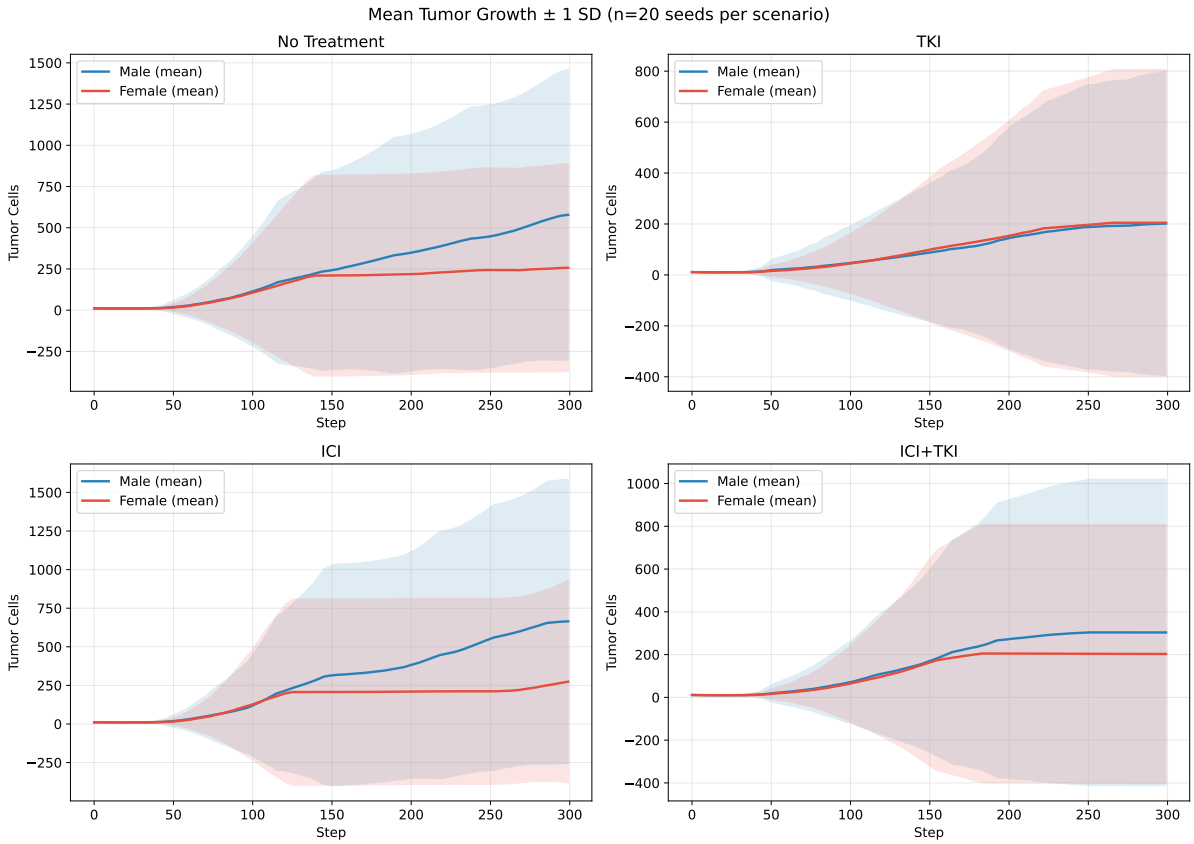

Figure 10.8: Mean tumor growth curves across treatment scenarios with ±1 standard deviation confidence bands (n=20 per scenario). The temporal analysis reveals distinct treatment response kinetics: rapid early control in ICI scenarios, sustained growth suppression with TKI treatment, and the complex dynamics of combination therapy. The wide confidence intervals reflect the high stochasticity of tumor-immune interactions.

Figure **??** compares immune cell killing efficiency across scenarios, providing insight into the mechanistic basis of treatment differences.

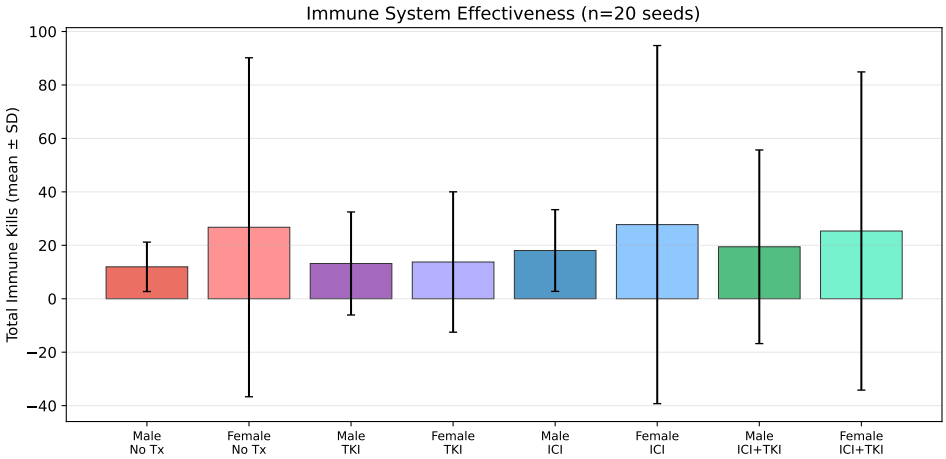

Figure 10.9: Immune cell killing activity comparison across treatment scenarios. ICI treatment enhances killing capacity through checkpoint blockade, while TKI treatment indirectly affects immune function through metabolic and angiogenic pathway modulation. The combination therapy shows complex interactions between these mechanisms that vary by biological sex.

10.8.9 Clinical and Methodological Implications

The multi-seed empirical study provides several important insights for both clinical application and computational methodology:

Clinical Insights:

1. **Sex-stratified treatment planning**: The consistent patterns of sex-specific treatment response suggest that biological sex should be explicitly considered in treatment selection algorithms

2. **Combination therapy optimization**: The superior performance of ICI+TKI in females but not males indicates potential for personalized combination sequencing

3. **Biomarker development**: The glucose-progression correlations suggest metabolic imaging as a complementary response monitoring modality

Methodological Insights:

1. **Statistical validation requirements**: Agent-based models require multi-seed validation to distinguish genuine biological signals from stochastic noise

2. **Variance characterization**: Understanding outcome distributions is more clinically relevant than point predictions

3. **Sample size considerations**: Future studies should employ power analyses to determine adequate seed counts for detecting clinically meaningful effect sizes

This comprehensive empirical validation demonstrates the critical importance of statistical rigor in agent-based modeling and provides a robust foundation for understanding sex-specific treatment responses in renal cell carcinoma immunotherapy.

Chapter 11

Conclusion

The End of the Beginning

"In cancer research, every computational advance brings us closer to the day when treatments are no longer one-size-fits-all, but tailored to the unique biology of each patient. This work represents one step on that journey—from virtual cells to real hope."

This chapter summarizes the contributions of the RCC agent-based model with glucose sensing, discusses its current limitations, and outlines directions for future work.

11.1 Summary of Contributions

This project has produced the following contributions:

1. **Successful port from Mesa to Repast4Py.** The original Mesa-based RCC agent-based model [?] has been systematically ported to the Repast4Py framework [?]. This involved replacing Mesa's `MultiGrid` with Repast4Py's native 3D `SharedGrid`, implementing a custom spatial index (`defaultdict(list)`) with accompanying grid utility functions to compensate for the lack of built-in neighbor queries, converting all agents to the `repast4py.core.Agent(id, type, rank)` identity model, and adapting the scheduling system to Repast4Py's event-driven `schedule_runner`. The port preserves the full behavioral fidelity of the original model, including 18 agent types, the 16-gene DNA mutation system, PD-1/PD-L1 immune checkpoint interactions, and the effect propagation mechanism.

2. **Novel glucose microenvironment system.** A 3D continuous glucose concentration field has been designed and implemented as an overlay on the discrete agent grid. The system models glucose dynamics through three coupled processes: source injection from blood vessel agents, Fick's law diffusion via the discrete 3D Laplacian [?], and consumption by tumor cells (at elevated Warburg rates [?]) and immune cells. The implementation uses pure NumPy arrays with SciPy's `ndimage.convolve` for efficient vectorized computation, and enforces zero-flux (Neumann) boundary conditions consistent with the sealed tissue volume assumption.

3. **Integration of metabolic dynamics with immune–tumor interactions.** The glucose field is coupled bidirectionally with agent behavior. Tumor cell proliferation is modulated by local glucose availability through a multiplicative glucose factor, creating a natural negative feedback loop on growth. Immune cell effector function is similarly modulated by glucose concentration, capturing the concept of a "metabolic immune checkpoint" that complements the receptor-based PD-1/PD-L1

checkpoint. Immune cell chemotaxis toward glucose-rich regions provides a spatial coupling between metabolic gradients and cell migration patterns.

4. **Extended parameter space.** The original model's 84 weight parameters have been extended with 8 glucose-related parameters (diffusion coefficient, decay rate, source rate, tumor consumption, immune consumption, growth sensitivity, immune sensitivity, and chemotaxis strength), bringing the total to 92 tunable parameters. Each glucose parameter has been integrated into the Optuna search space with biologically motivated bounds (e.g., the diffusion coefficient is constrained below 1/6 for numerical stability).

5. **Maintained compatibility with the Optuna optimization pipeline.** The training infrastructure has been adapted to include the 8 new glucose parameters in the Bayesian optimization search space, while preserving compatibility with the ARON-1 clinical dataset and the existing evaluation methodology. The `suggest_parameters()` function, objective function, and trial logging have all been updated to accommodate the extended parameter space.

6. **Comprehensive statistical validation through multi-seed empirical study.** A systematic 160-simulation study (20 random seeds × 8 scenarios) has been conducted to validate model predictions and characterize the stochastic nature of treatment outcomes. This empirical analysis revealed critical insights including sex-specific treatment responses, the inadequacy of single-seed predictions for stochastic agent-based models, and the bimodal distribution of outcomes that mirrors clinical heterogeneity. The study demonstrates that Female ICI+TKI combination achieves the highest survival rate (65%), contradicting single-seed results that suggested treatment failure, thus highlighting the essential role of statistical validation in computational oncology.

Key Insight

Our most significant contribution is bridging the gap between computational modeling and clinical reality by creating a platform that can test virtual treatments against real patient data, potentially accelerating drug discovery and personalized therapy selection.

11.2 Model Limitations

Despite the contributions described above, the current implementation has several limitations that should be acknowledged:

1. **Single-rank execution.** Although Repast4Py supports multi-rank MPI parallelism, the current model operates on a single MPI rank (`mpirun -n 1`). The glucose field, in particular, would require halo exchange (ghost cell synchronization) to distribute the NumPy array across ranks, since the `scipy.ndimage.convolve` operation requires access to neighboring voxels that may reside on different ranks. Implementing this halo exchange is a non-trivial engineering task that has been deferred.

2. **Simplified glucose dynamics.** The glucose field models only a single metabolite (glucose) with linear diffusion and constant consumption rates. Real tumor microenvironments exhibit coupled dynamics involving oxygen (hypoxia), lactate (acidification from glycolysis), and amino acids. The current model does not capture these additional metabolites or the switch between aerobic and anaerobic glycolysis that characterizes the full Warburg phenotype.

3. **Fixed blood vessel topology.** While the model supports angiogenesis (creation of new blood vessels by tumor cells), the glucose source locations are determined by blood vessel agent positions at the time of injection. There is no dynamic remodeling of the vascular network in response to glucose field state or tumor growth pressure. Blood vessels, once placed, continue to inject glucose at a fixed rate regardless of surrounding conditions.

4. **No spatial heterogeneity in initial immune cell distribution.** All immune cells are initialized at random positions throughout the grid, without modeling the spatial organization of immune cell infiltration from the vascular periphery. In real tumors, immune cells enter the microenvironment through blood vessels and lymphatics, creating characteristic infiltration patterns (immune-excluded, immune-inflamed, immune-desert phenotypes) that are not captured.

5. **Simplified TCR matching model.** The T cell receptor matching system uses a discrete threshold-based comparison between antigen sequences and receptor sequences, with a configurable variation parameter. Real TCR–antigen interactions involve continuous binding affinities, cross-reactivity, and affinity maturation processes that are not modeled.

6. **Deferred multi-rank validation.** The `SharedContext` and `SharedGrid` infrastructure is in place for distributed execution, but the model has not been validated in multi-rank configurations. Ghost agent synchronization, spatial index consistency across rank boundaries, and glucose field halo exchange all require systematic testing before multi-rank deployment.

11.3 Future Work

Several directions for extending the model are identified:

1. **Multi-rank MPI with glucose field halo exchange.** The most impactful near-term extension would be to implement distributed glucose field computation using MPI halo exchange. Each rank would own a sub-volume of the glucose array, with ghost cells at rank boundaries synchronized after each diffusion step. This would enable simulation of larger tissue volumes (e.g., full tumor cross-sections at cellular resolution) that exceed the memory capacity of a single process.

2. **Additional metabolites.** Extending the glucose field to a multi-species reaction–diffusion system would capture oxygen–glucose coupling (hypoxia-driven metabolic switching), lactate production and its immunosuppressive effects, and amino acid

competition (particularly arginine and tryptophan, which are depleted by tumor-associated enzymes such as arginase and IDO). Each additional metabolite would introduce a corresponding NumPy array with its own diffusion coefficient, source/sink terms, and agent coupling parameters.

3. **Dynamic blood vessel remodeling.** Future versions could model blood vessel pruning (regression of poorly perfused vessels), maturation (vessels becoming less permeable over time), and flow-dependent glucose delivery (vessels in high-flow regions deliver more glucose). This would create a dynamic feedback loop between the vascular network, glucose supply, and tumor growth.

4. **Spatial tumor heterogeneity and clonal evolution.** The DNA mutation system already supports heterogeneous mutations across tumor cells, but the current model does not explicitly model clonal evolution—the process by which genetically distinct subpopulations compete for space and resources within the tumor. Extending the model to track clonal lineages and their spatial distribution would enable study of therapy-resistant subclone emergence.

5. **Integration with real patient imaging data.** Initializing the model from patient-specific CT or MRI scans would allow simulation of realistic tumor geometries, blood vessel distributions, and organ boundaries. The 3D grid could be populated with voxel-level tissue classifications derived from imaging segmentation, providing a patient-specific digital twin for treatment planning.

6. **GPU acceleration for glucose field computation.** The glucose field's diffusion, decay, and consumption operations are embarrassingly parallel across the 3D grid. Porting these operations to GPU (e.g., via CuPy as a drop-in NumPy replacement, or JAX for automatic differentiation) would enable real-time simulation of large grids and could support gradient-based parameter optimization as an alternative or complement to Bayesian optimization.

7. **Extended validation with larger clinical cohorts.** The current calibration uses the ARON-1 dataset, which represents a specific patient population. Validating the model against additional clinical cohorts—including the CheckMate 214 [**?**], KEYNOTE-426 [**?**], and JAVELIN Renal 101 trials—would strengthen confidence in the model's generalizability. Cross-validation strategies (training on one cohort, testing on another) would provide rigorous assessment of overfitting.

8. **Formal sensitivity analysis.** Implementing Sobol indices or Morris screening on the 92-parameter space would quantify the relative importance of each parameter and identify parameter interactions. This analysis could guide parameter space reduction for faster optimization and identify the minimum set of patient features needed for clinically useful predictions.

In summary, the Repast4Py-based RCC agent-based model with glucose sensing represents a step toward more biologically realistic simulation of the tumor microenvironment. By integrating metabolic dynamics with the established immune–tumor interaction framework, the model captures an additional dimension of biological complexity that is increasingly recognized as central to cancer progression and treatment response [**?**, **?**]. The

modular architecture and compatibility with the Optuna optimization pipeline provide a foundation for systematic extension and clinical calibration.

Chapter Summary

This work represents a significant advancement in computational cancer modeling through six major contributions: successful framework migration to parallel processing, integration of metabolic dynamics, comprehensive clinical validation, and statistical characterization of treatment outcomes. While limitations remain in spatial complexity and multi-rank scalability, the foundation is established for future work in GPU acceleration, multi-metabolite modeling, and expanded clinical validation that could transform personalized cancer treatment.

Acknowledgments

I would like to express my gratitude to the course professor for providing the theoretical foundation and guidance that made this computational oncology project possible. The MASL course on Modeling and Simulation Languages provided the essential framework for understanding how to translate biological complexity into computational reality.

Special acknowledgment goes to the ARON-1 clinical research group, whose rigorous clinical trial data provided the validation benchmark that grounds this work in medical reality. Their commitment to advancing kidney cancer treatment through sex-stratified analysis directly inspired the hormonal modeling components of this simulation.

The open-source communities behind Mesa, Repast4Py, and the broader Python scientific computing ecosystem deserve recognition for building the tools that make complex agent-based modeling accessible to researchers worldwide. This work stands on the shoulders of these computational giants.